

Universität Siegen
Naturwissenschaftlich-Technische Fakultät
Department Elektrotechnik und Informatik
Lehrstuhl für Regelungs- und Steuerungstechnik

Studienarbeit

Design and Fabrication of Remotely Operated Two-Wheeled Self-Balancing Robot with Integrated ToF Sensor Array for 2D Mapping

by

Muhammad Haris Mujeeb

Matriculation number

1721876

Examiner

Prof. Dr.-Ing. habil. Michael Gerke

2nd Examiner

Dipl.-Ing. Nasser Gyagenda

Date of delivery

12.05.2025

Abstract—This report presents the design and implementation of a remotely controlled two-wheeled self-balancing robot with TeraRanger Multiflex, consisting of eight Time-of-Flight (ToF) sensors for two-dimensional (2D) environmental mapping. The system architecture comprises a differential-drive mobile base, a ToF sensors array for spatial perception and an embedded communication and control hub for data integration and external interfacing. The mobile base consists of a pair of DC motors, an Arduino Nano microcontroller, an Inertial Measurement Unit (IMU) and Hall-effect rotary encoders to provide accurate state estimation. A Multi-rate Proportional-Integral-Derivative (PID) control algorithm is employed to ensure stable balancing and precise locomotion control.

The ESP32-S3-DevKitC-1 serves as the central communication and control hub, enabling efficient data exchange between the two-wheeled mobile base, the ToF sensor array, and external systems such as computers, microcontrollers, mobile devices, or cloud services. This setup facilitates real-time monitoring and remote control via Wi-Fi, thereby enhancing the system's adaptability across various applications. The firmware for both the Arduino and ESP32-based microcontrollers was developed using PlatformIO and Arduino IDE. The robot continuously transmits telemetry and ToF sensors data while receiving commands through Wi-Fi. The complete project, including source code and detailed documentation, is available at the following GitHub repositories:

- <https://github.com/haris-mujeeb/Self-Balancing-Robot>
- <https://github.com/haris-mujeeb/TeraRanger-Multiflex-DEMO>
- https://github.com/haris-mujeeb/TeraRanger_TwinDrive

Contents

1	Introduction	1
2	Project Objectives	2
3	Literature Review	2
4	Mathematical Modelling	4
4.1	State Equations	4
4.2	Controllability and Observability	6
5	Implementation	7
5.1	Software Implementation	7
5.2	Hardware Implementation	8
5.2.1	Microcontrollers	9
5.2.2	Power Supply	9
5.2.3	Inertial Measuring Unit	11
5.2.4	Motor Driver IC	12
5.2.5	Inverted Schmitt Trigger	12
5.2.6	DC Drive Motors	13
5.2.7	Infrared Sensing	14
5.2.8	Ultrasonic Distance Sensor	14
5.2.9	Bluetooth Module Integration	14
5.3	TeraRanger Multiflex	15
6	Sensor Fusion	16
6.1	Complementary Filter	16
6.2	Extended Kalman Filter	16
6.2.1	Extended Kalman Filter for Estimating Pitch Angle	17
6.2.2	Predicted State Estimate	18
6.2.3	Predicted Error Covariance	18
6.2.4	Innovation Residual	18
6.2.5	Innovation Covariance	18
6.2.6	Optimal Kalman Gain	19
6.2.7	Corrected State Estimate	19
6.2.8	Corrected Estimate Covariance	19
6.3	Software Implementation of EKF	19
7	Motor Control	21
7.1	PID Controller	21
7.2	PID Tuning Methodology	22
7.2.1	Ziegler-Nichols Method	22
7.2.2	Practical Tuning Guidelines	23
7.3	Multi-rate PID Control	23

7.3.1	Pitch Control	23
7.3.2	Yaw Control	26
7.3.3	Position Control	27
7.3.4	Combining Control Outputs	28
8	Front Obstacle Detection	30
8.1	Operation of Ultrasonic sensor	30
8.1.1	Sound Signal Transmission	30
8.1.2	Sound Signal Reception	31
8.2	Infrared Sensing	32
8.2.1	IR Signal Emission	32
8.2.2	IR Signal Reception	32
8.2.3	Binary Filter:	33
8.3	Obstacle Decision Logic	35
9	Communication Hub	36
9.1	Communication with the Mobile Base	36
9.1.1	Sending Commands	36
9.1.2	Receiving Telemetry Data	37
9.2	Communication with TerraRanger Multiflex	38
9.3	Communication with External Systems	39
9.3.1	Telemetry Transmission	39
9.3.2	Remote Command Reception and Execution	39
10	Conclusion	41
11	Future Work and Improvements	42
	Appendices	43
A	Calculation of the system's transfer functions	43
A.1	Equation for the cart	43
A.2	Equation for the the pendulum	43
A.3	Combining the equations	44
A.4	Linearising the equations	44
A.5	Laplace transform	44
A.6	State Space Modelling	45
B	Calculation for Kalman Filter	46
B.1	Time-Discrete Kalman Filter Model	47
B.2	Implementing Kalman Filter	49
B.2.1	Predicted State Estimate	49
B.2.2	Predicted Error Covariance	49
B.2.3	Innovation Residual	49
B.2.4	Innovation Covariance	50

B.2.5	Optimal Kalman Gain	50
B.2.6	Corrected State Estimate	50
B.2.7	Corrected Estimate Covariance	50

List of Figures

Fig. 1	Forecast of the MEMS market value in billions of US dollars by year. Reproduced from an open-access source [3].	1
Fig. 2	ELEGOO Tumbler used as the mobile base for this project [7].	3
Fig. 3	TeraRanger Multiflex circular mount [18].	3
Fig. 4	ESP32-S3-DevKitC-1-N8R8 from Espressif Systems[19].	3
Fig. 5	Simplified inverted pendulum model for the balancing robot [8].	4
Fig. 6	Pole-zero map of the open loop system G_ψ revealing a pole in the right half plane.	5
Fig. 7	A simplified block diagram of the system architecture.	7
Fig. 8	A simplified flow diagram of the system architecture.	8
Fig. 9	(a) ATmega328p microcontroller by Microchip [21]; (b) Arduino Nano board by Arduino [22]	9
Fig. 10	ELEGOO chargeable battery pack.	10
Fig. 11	Schematic of the voltage divider circuit utilised in this project. The voltage VM is connected to the positive terminal of the battery, while VOL is connected to the Arduino Nano A2 pin [7].	10
Fig. 12	MPU-6050 6-axis IMU [24].	11
Fig. 13	(a) TB6612FNG motor driver [26]; (b) SN74LVC2G14 dual Schmitt-trigger inverter [27].	12
Fig. 14	Target characteristics for TB6612FNG [25].	12
Fig. 15	Schmitt trigger output without hysteresis (left) and with hysteresis (right) [28].	13
Fig. 16	Using SN74LVC2G14 in conjunction with TB6612FNG to reduce MCU pin usage and noise filtering [7].	13
Fig. 17	NHYTech GA37 520 DC motor [29].	13
Fig. 18	(a) IR204C-A infrared LED [31]; (b) IRM-56384 infrared reciever [32].	14
Fig. 19	(a) HC-SR04 ultrasonic distance sensor [30]; (b) Bluetooth module [33].	14
Fig. 20	Pitch angle calculated using a complementary filter with $\alpha = 0.9$. It can be observed that the high frequency accelerometer noise (circled in red) are filtered properly. And similarly the low frequency integration errors of the gyroscope (red) are removed over longer interval of time.	17
Fig. 21	Block diagram for pitch angle ψ control.	21
Fig. 22	A simplified flow diagram of the multi-rate PID control loop used.	24
Fig. 23	Output values calculated by the pitch angle PD controller at $K_{p\psi} = 55$ and $K_{d\psi} = 1.25$. The data was recorded at baud rate of 115200.	25
Fig. 24	Output values calculated by the pitch angle PD controller at $K_{p\psi} = 55$ and $K_{d\psi} = 0.75$. Data recorded at baud rate of 115200.	25
Fig. 25	Output values calculated by the pitch angle PD controller with an adaptive derivative gain at $K_{p\psi} = 55$ and $K_{d\psi} = 0.75 \sim 1.25$. The data was recorded at baud rate of 115200.	26
Fig. 26	Yaw angle control system response at $K_p = 2.5$ and $K_d = 0.5$ for angle varying from -20 deg to $+20 \text{ deg}$. The data was recorded at baud rate of 115200.	27

Fig. 27	Position control system response with $K_{px} = 0.26$ and $K_{dx} = 20$ for a displacement of -600 mm to $+300\text{ mm}$. Data recorded at baud rate of 115200.	29
Fig. 28	A simplified diagram of the motor control loop [35].	29
Fig. 29	The received raw data from the TerraRanger Multiflex via UART.	38
Fig. 30	Real-time sensor data visualization in a polar plot.	39
Fig. 31	Final assembly of the remotely operated two-wheeled robot with TerraRanger Multiflex.	41
Fig. 32	Free-body diagram of the simplified system with cart and pendulum [35]. Where, x axis is along the horizontal axis and y axis is along the vertical axis and $\psi = \pi - \theta$	43

List of Tables

Tab. 1	UART Communication Configuration	15
Tab. 2	I2C Communication Methods	36
Tab. 3	UART Communication Methods	37
Tab. 4	List of Commands and Corresponding Values	37

1. Introduction

The World Robotics report reveals a record 4.28 million operational industrial robots globally in 2023, with Asia leading installations at 70%, driven primarily by China’s significant market share, while overall demand for robots is expected to stabilize and grow in the coming years despite geopolitical uncertainties [1]. The mobile robot market, particularly for autonomous mobile robots (AMRs), is experiencing rapid growth across industrial and commercial sectors, with the overall AMR market projected to grow at 24% annually—from \$19 billion in 2018 to \$54 billion by 2023 [2]. Driven initially by automotive and electronics sectors, demand is now expanding into logistics, healthcare, and consumer goods, with Europe leading the market and significant contributions from the USA and China [2].

Over the past decade, advancements in Micro-Electro-Mechanical Systems (MEMS) and embedded computing have revolutionized the design and functionality of robots. Membrane-based MEMS, such as pressure sensors, radio frequency MEMS, accelerometers, microphones, gyroscopes and inertial combos are commercially the most relevant devices by producing the most revenue as shown in Fig. 1.

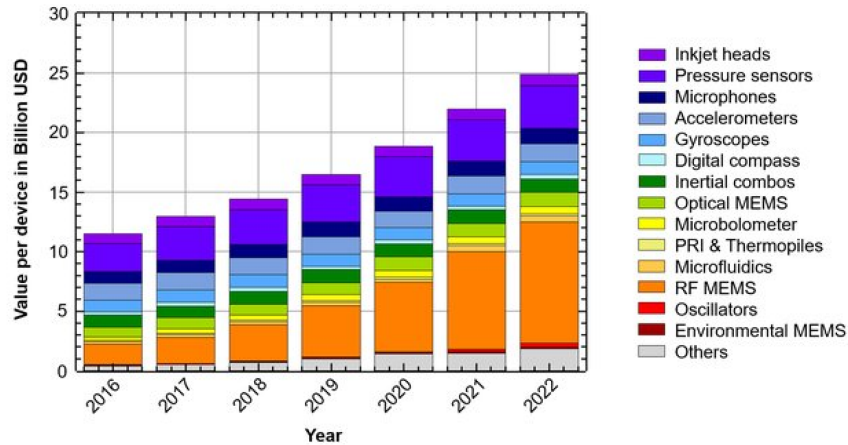


Figure 1: Forecast of the MEMS market value in billions of US dollars by year. Reproduced from an open-access source [3].

The worldwide MEMS market is expected to grow to \$64.15 billion by 2035 [4]. These advancements in MEMS and embedded computing have also revolutionized the design and functionality of mobile robots. For instance, the introduction of gyros, accelerometers, and other sensors has enabled these robots to achieve higher levels of balance and stability. One interesting area of research is the integration of cost effective Time-of-Flight (ToF) sensors, a type of Optical MEMS sensors, with mobile base. These sensors are particularly useful for applications requiring 2D mapping, such as navigation, localization, and obstacle detection.

Therefore, research into the design and fabrication of a low-cost, user-friendly mobile robot base equipped with ToF sensors is both timely and strategic. Such a platform would provide accessible entry points for education, research, and small-to-medium enterprises looking to adopt robotic automation, bridging the gap between high-end industrial solutions and affordable, scalable alternatives.

2. Project Objectives

The primary objectives of this project are:

- To design and implement a two-wheeled self-balancing mobile platform capable of maintaining dynamic stability using real-time sensor feedback and control algorithms.
- To develop a robust control framework that ensures precise balancing and smooth motion control, with resilience to external disturbances.
- To integrate an Inertial Measurement Unit (IMU) and Hall-effect rotary encoders, utilizing sensor fusion techniques such as the Kalman filter, to achieve accurate state estimation, including orientation and velocity.
- To enhance the robot's spatial awareness through 2D environmental mapping using Time-of-Flight (ToF) sensors.
- To develop a central communication and control hub, facilitating data integration between the mobile base, ToF sensors, and external systems, enabling real-time monitoring and remote control over Wi-Fi.
- To provide comprehensive documentation of the system design, control strategies, and software implementation, and to openly share the source code and resources via GitHub to support reproducibility and future development.

3. Literature Review

Wheeled robots are inherently more efficient than bipedal counterparts due to their lower energy consumption, simpler mechanics, and reduced control complexity. This makes them not only more cost-effective to produce and maintain but also highly suitable for structured indoor environments where agility and compactness are essential [5]. Today, robots are used in a variety of environments beyond industrial and clinical settings, including everyday locations such as offices, hotels, hospitals, and homes, where they frequently interact with people. In these situations, mobile robots frequently need to manoeuvre effectively within limited spaces [6]. Therefore, a two-wheeled robot was selected for the mobile base for this research project. The ELEGOO Tumbler (shown in Fig. 2) was selected as a base platform, as it provides nearly all the necessary hardware in a convenient DIY kit.

Two-wheeled self-balancing robots have been widely studied as a variant of the classic inverted pendulum problem, requiring control strategies to maintain stability. Various approaches, including Proportional-Integral-Derivative (PID) control [8], Lead-Lag [9] and Fuzzy Control [10] have been explored to achieve robust balancing and motion control. Abdelgawad et al. [11] demonstrated that PID control offers higher accuracy, lower error, faster response, and greater robustness, while also being easier to tune—making it well-suited for precise control applications. Kalman filter [12] has been shown to significantly enhance state estimation accuracy [13], [14], [15], and is therefore also employed in this project.

This project, we aim to develop a lightweight and efficient system for 2D environmental mapping in compact indoor settings. For Time-of-Flight (ToF) sensing, the Terabee TeraRanger Multiflex (shown



Figure 2: ELEGOO Tumbler used as the mobile base for this project [7].

in Fig. 3) was selected due to its reliable range of up to 2 meters and its modular design featuring eight pre-calibrated, plug-and-play sensors [16]. These characteristics make it particularly well-suited for applications that require low power consumption and scalability. The ESP32-S3-DevKitC-1 (shown in Fig.4) was similarly selected as the communication hub for its open-source support, integrated Wi-Fi, and dual-core processing [17]. These capabilities enable efficient real-time data handling and peripheral communication, which are critical for processing input from the sensors and forwarding.



Figure 3: TeraRanger Multiflex circular mount [18].

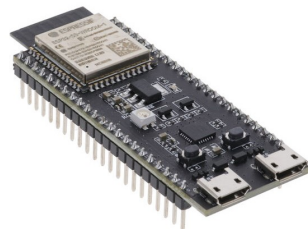


Figure 4: ESP32-S3-DevKitC-1-N8R8 from Espressif Systems[19].

4. Mathematical Modelling

The balancing robot is typically represented as a rigid rod attached to a joint, mounted on a rigid cart that moves in a single direction. For simplification, the wheelbase of the robot is assumed to behave like a cart sliding on a frictionless surface, as shown in Fig. 5. This modelling approach follows Prasad and Tyagi [20], and the MathWorks [8].

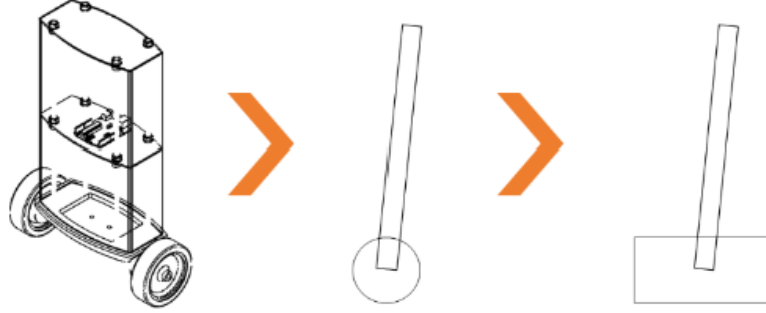


Figure 5: Simplified inverted pendulum model for the balancing robot [8].

4.1. State Equations

The state space form of the robot is given by,

$$\begin{bmatrix} \ddot{x} \\ \ddot{\psi} \end{bmatrix} = A \begin{bmatrix} x \\ \dot{\psi} \end{bmatrix} + B u_{input} \quad (1)$$

$$y = C u_{input} \quad (2)$$

Using the free-body diagram for the simplified inverted pendulum model, as illustrated in Fig. 32, the system matrices A , B and C were derived as,

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & \frac{-(I_{pend} + m_{pend}l^2)f}{I_{pend}(m_{cart} + m_{pend}) + m_{cart}m_{pend}l^2} & \frac{m_{pend}^2gl^2}{I_{pend}(m_{cart} + m_{pend}) + m_{cart}m_{pend}l^2} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{-m_{pend}lf}{I_{pend}(m_{cart} + m_{pend}) + m_{cart}m_{pend}l^2} & \frac{m_{pend}gl(m_{cart} + m_{pend})}{I_{pend}(m_{cart} + m_{pend}) + m_{cart}m_{pend}l^2} & 0 \end{bmatrix} \quad (3)$$

$$B = \begin{bmatrix} 0 \\ \frac{I_{pend} + m_{pend}l^2}{I_{pend}(m_{cart} + m_{pend}) + m_{cart}m_{pend}l^2} \\ 0 \\ \frac{m_{pend}l}{I_{pend}(m_{cart} + m_{pend}) + m_{cart}m_{pend}l^2} \end{bmatrix} \quad (4)$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (5)$$

For complete derivation of A , B and C , see Appendix A. After Laplace transformation, linearisation and combining the equation of motion for the pendulum (Eq. 38) and equation of motion for the cart (Eq. 39), we get the transfer function $G_\psi(s)$ for vertical angle of the robot $\Psi(s)$,

$$\Psi(s) = \frac{\frac{m_{pend}l}{q}s}{\underbrace{s^3 + \frac{f(I_{pend}+m_{pend}l^2)}{q}s^2 - \frac{(m_{cart}+m_{pend})m_{pend}gl}{q}s - \frac{fm_{pend}gl}{q}}_{G_\theta(s)}} U_{input}(s)$$

and the transfer function $G_x(s)$ horizontal cart position $X(s)$,

$$X(s) = \frac{\frac{(I_{pend}+m_{pend}l^2)s^2 - gm_{pend}l}{q}}{\underbrace{s^4 + \frac{f(I_{pend}+m_{pend}l^2)}{q}s^3 - \frac{(m_{cart}+m_{pend})m_{pend}gl}{q}s^2 - \frac{fm_{pend}gl}{q}s}_{G_x(s)}} U_{input}(s)$$

Where f is friction coefficient, l is the distance to the centre of mass, I_{pend} is the moment of inertia of the pendulum, m_{cart} is the mass of the cart, m_{pend} is the mass of the pendulum, g is the acceleration due to gravity and q is defined as $q = (m_{cart} + m_{pend})(I_{pend} + m_{pend}l^2) - (m_{pend}l)^2$, as illustrated in Fig. 32.

Using MATLAB, pole zero map for the open loop system G_ψ can be generated (see Fig. 6). It is concluded that for all positive value of f , l , I_{pend} , m_{cart} , m_{pend} and g the system in itself is unstable since it has a pole on the positive real axis as can be seen in Figure 6.

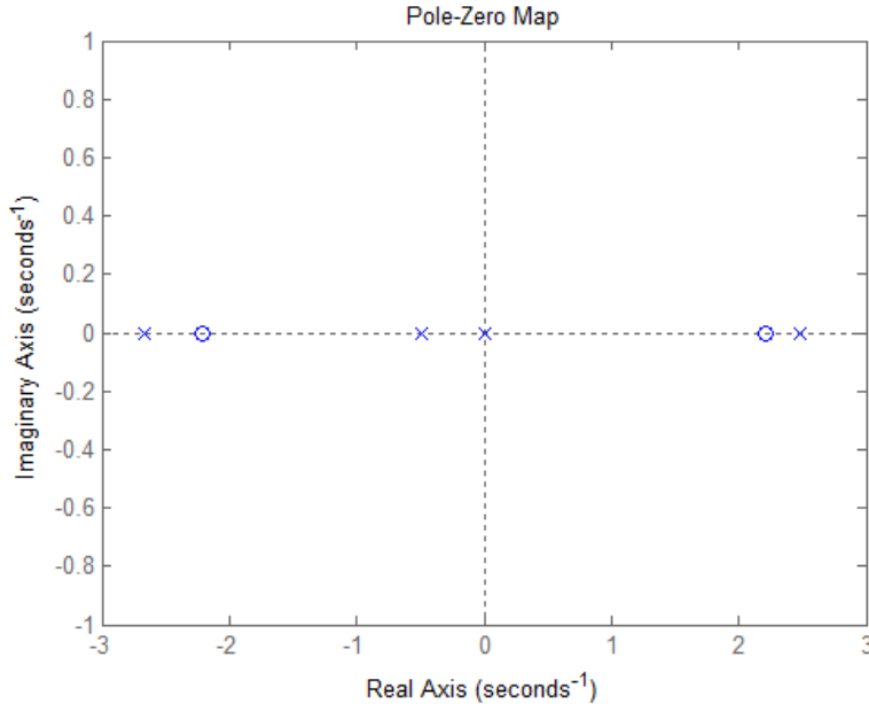


Figure 6: Pole-zero map of the open loop system G_ψ revealing a pole in the right half plane.

4.2. Controllability and Observability

Controllability ensures that the system can be driven from any initial state to any desired state using a suitable control input. Observability guarantees that the system's internal states can be reconstructed from its outputs. These properties are determined by the rank of the controllability matrix S and the observability matrix O , respectively. If both matrices S and O have full rank, then the system is fully controllable and observable.

$$S = \begin{bmatrix} B & AB & A^2B & A^3B \end{bmatrix} \quad (6)$$

$$O = \begin{bmatrix} C \\ CA \\ CA^2 \\ CA^3 \end{bmatrix} \quad (7)$$

Using the system matrices A , B and C (see equations 49, 50 and 51 in Appendix A), the rank of S and O confirms that the system is controllable and observable for any positive value of l , I_{pend} , m_{cart} , m_{pend} and g .

5. Implementation

The system architecture of the robot can be divided into three parts Mobile Base, TerraRanger Multiflex Hub and Communication Hub as shown in Fig. 7. This robot can also communicate with external systems such as computers, microcontrollers, mobile devices, or cloud services via Wi-Fi.

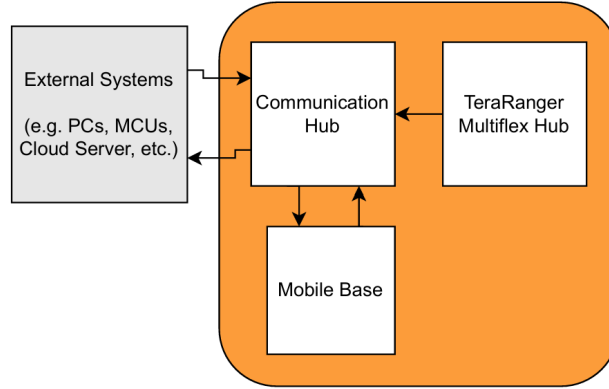


Figure 7: A simplified block diagram of the system architecture.

5.1. Software Implementation

The Figure 8 illustrates the overall system architecture and control flow for the self-balancing mobile base. The process begins with system initialisation and calibration of its various sensors. Sensor data, including encoder values, IMU data, and obstacle distance, is acquired and passed through respective filters (Weighted Moving Average (WMA), Kalman, and Binary Counting) to reduce noise and improve accuracy. The filtered sensor data is then aggregated and used to update the robot’s internal state. Subsequently, the user commands are received, parsed and processed.

To stabilize the inverted pendulum in the upright position and to control the robot at the desired position and yaw angle, three PID controllers, each dedicated to managing one of these parameters. The PID control algorithm (shown in Fig. 22) utilises the updated sensor data to calculate appropriate motor commands. These commands are then sent to the motors, driving the robot’s movement. Throughout this process, telemetry data is generated and transmitted, providing feedback on the robot’s status and performance. The refined distance data is then transmitted to the Communication Hub via a UART interface.

TeraRanger Multi-flex Hub reads the raw ToF measurements from the sensors array and processes them using data onboard filtering algorithms to improve accuracy. This processed data is then send to the Communication Hub.

The Communication Hub acts as the central gateway, relaying sensor data to external systems (e.g., computers, cloud services) and forwarding control commands from these systems to the mobile base. It employs a bidirectional Wi-Fi protocol to ensure low-latency data exchange, facilitating real-time remote supervision and intervention when necessary.

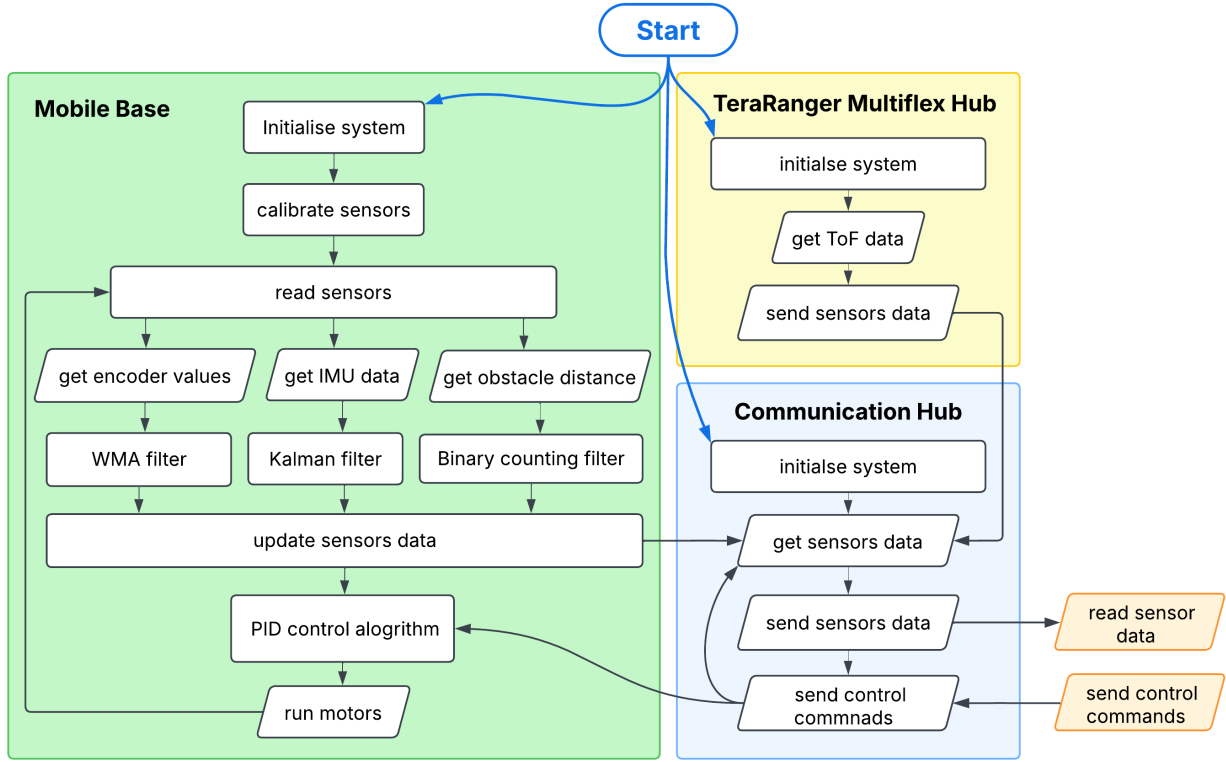


Figure 8: A simplified flow diagram of the system architecture.

5.2. Hardware Implementation

The system integrates a modular hardware architecture centered on the Arduino Nano (ATmega328P) for low-level motor control and sensor interfacing, complemented by the ESP32-S3-DevKitC-I as a high-performance communication hub. Key components include:

Sensing and Perception:

- MPU6050 (6-axis IMU) for real-time attitude estimation via fused accelerometer/gyroscope data.
- HC-SR04 ultrasonic and infrared (IR) sensors for front obstacle detection, enabling reactive navigation.
- TeraRanger Multi-flex for 2D mapping, interfaced via UART.

Actuation and Control:

- TB6612FNG motor driver for driving DC gear motors with Hall-effect encoders.
- PID control loops (running on the Arduino Nano) for balancing, rotation and position control.

Communication:

- BT16 Bluetooth module for short-range telemetry and configuration.
- ESP32-S3-DevKitC-I managing Wi-Fi/Bluetooth LE connectivity, enabling remote monitoring and cloud integration.

Power Management:

- Two 18650 Lithium Polymer (LiPo) batteries (7.4 volts nominal) with integrated BMS for overcharge/discharge protection, supplying regulated 5 volts rails to peripherals.

5.2.1. Microcontrollers

This project utilizes two microcontrollers: the ATmega328P and the ESP32-S3. The ATmega328P, integrated into the Arduino Nano (Fig. 9), serves as the primary controller for the two-wheeled mobile base. It handles real-time tasks such as sensor data acquisition, filtering, motion control, and basic communication. With a 16 MHz clock, 32 KB of flash memory, 2 KB SRAM, and 1 KB EEPROM [21], it provides sufficient resources for embedded control tasks in this application. The Arduino Nano, based on the ATmega328P, was chosen for its compact form factor, low cost, and strong community support, making it ideal for prototyping and rapid integration with sensors, encoders, and motor drivers [22]. It communicates with the ESP32-S3 via UART at 9600 bps, transmitting control commands and telemetry data.

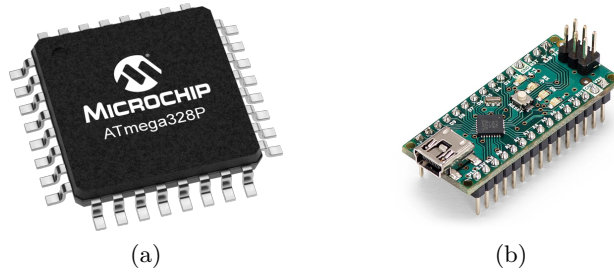


Figure 9: (a) ATmega328p microcontroller by Microchip [21]; (b) Arduino Nano board by Arduino [22]

The ESP32-S3, integrated into ESP32-S3-DevKitC-1 (Fig. 4), serves as the central communication and processing hub. It interfaces with the TeraRanger Multiflex Hub over a separate UART channel at 115200 bps, enabling high-speed data transfer from the ToF sensor array. Additionally, its onboard Wi-Fi capability allows remote access and real-time data exchange with external systems—such as computers, microcontrollers, or cloud services—using an Access Point (AP) configuration.

Both the ESP32-S3 and the TeraRanger Multiflex are powered by the same battery supplying the mobile base, streamlining the power architecture and ensuring reliable operation in mobile scenarios.

5.2.2. Power Supply

The custom-designed battery box, illustrated in Fig. 10, offers a portable and rechargeable power solution for the Elegoo robot [23]. It accommodates two 18650 LiPo batteries, configured in series to provide a regulated nominal voltage of 7.4 volts, which is suitable for powering all robot components, including the mobile base, the TeraRanger Multiflex Hub and the ESP32-S3-DevKitC-1. An integrated Battery Management System (BMS) ensures safe operation by protecting against overcharging, over-discharging, over-current conditions, and short circuits. Additionally, a power switch enables complete disconnection of the power supply, while a USB charging port and a status indicator LED facilitate convenient recharging and monitoring of the battery's status.



Figure 10: ELEGOO chargeable battery pack.

The battery voltage level can be monitored using 10-bit ADC available in ATmega328P [21]. But ATmega328P works within the voltage range of 0-5 volts. For this reason, a voltage divider circuit is employed, as shown in Fig. 11.



Figure 11: Schematic of the voltage divider circuit utilised in this project. The voltage VM is connected to the positive terminal of the battery, while VOL is connected to the Arduino Nano A2 pin [7].

The voltage divider circuit, depicted in Fig. 11, comprises of two resistors connected in series. The output voltage is expressed as follows:

$$V_{\text{out}} = V_{\text{battery}} \times \frac{R_2}{R_1 + R_2} \quad (8)$$

Where V_{out} is the scaled-down voltage read by the ADC of the Arduino Nano, while R_1 and R_2 are the resistor values selected to ensure the measured voltage remains within the 0–3.3 volts range. Specifically, a series configuration of 10 k Ω (R_1) and 1.5 k Ω (R_2) resistors is utilised, resulting in a voltage division ratio of approximately 1:11.

$$V_{\text{out}} = V_{\text{battery}} \times \frac{1.5}{10 + 1.5} = V_{\text{battery}} \times 0.136 \quad (9)$$

For a fully charged 8.4 V LiPo battery, the resulting output voltage V_{out} is approximately 1.1 volts. This adjustment aligns the measured voltage more closer with the internal reference voltage of 1.1 volts, as specified in the data-sheet [21]. This ensures stable readings independent of supply voltage fluctuations, but may introduce minor ADC saturation at peak voltage.

An algorithm, similar to the one illustrated in Alogrithm 1, is employed that periodically measures the voltage at Arduino Nano pin A2 and sends an error message "Low battery" if it falls below a defined threshold.

Algorithm 1 Voltage Monitoring and Motor Shutdown

```
1: procedure CHECKVOLTAGELEVEL(lastCheckTime)
2:   if current time – lastCheckTime > VOLTAGE_CHECK_INTERVAL then
3:     Read voltageValue from ADC
4:     Update lastCheckTime to current time
5:     if voltageValue ≤ MINIMUM_ALLOWED_VOLTAGE then
6:       Stop motors
7:       Display error message: “Voltage low!”
8:       Set robotState ← ERROR
9:     end if
10:  end if
11: end procedure
```

5.2.3. Inertial Measuring Unit

The MPU6050 is a widely used 6-axis Inertial Measurement Unit (IMU) that integrates a three-axis gyroscope and a three-axis accelerometer on a single chip, making it essential for motion tracking and stabilization applications (shown in Fig. 12). Its compact design and built-in Digital Motion Processor (DMP) enable real-time processing of sensor data, which is crucial for robotics, drones, and wearable devices. In applications like self-balancing robots, it provides accurate orientation and acceleration data necessary for maintaining stability.

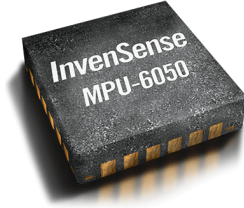


Figure 12: MPU-6050 6-axis IMU [24].

The MPU6050 is an IMU that provides the following data:

- Accelerometer Data: Measures acceleration in X, Y, and Z axes. It helps in estimating tilt based on gravitational force.
- Gyroscope Data: Measures angular velocity around X, Y, and Z axes. Integration of gyroscope readings over time provides the tilt angle.

However, the raw sensor data from an IMU presents several challenges:

- Accelerometer Noise: Although accelerometers provide an absolute angle reference, their readings are often noisy and susceptible to external forces, which can affect accuracy.
- Gyroscope Drift: Over time, integration errors accumulate, leading to drift and resulting in inaccurate angle estimations.

To address these issues, sensor fusion techniques are employed, combining data from both the accelerometer and gyroscope to enhance accuracy and stability. These techniques are discussed in more detail in Section 6.

5.2.4. Motor Driver IC

The TB6612FNG dual motor driver, as shown in Fig. 13a, allows independent control of two DC motors. It uses a MOSFET H-bridge for bidirectional and Pulse Width Modulation (PWM) based motor speed control. Fig. 14 shows that each channel of the TB6612FNG can deliver up to 0.85A of current continuously. Furthermore, it consists of integrated over-current protection and thermal shutdown features for enhance reliability [25].

5.2.5. Inverted Schmitt Trigger

To simplify direction control, an inverted Schmitt trigger allows a single MCU pin to generate both the control signal and its complement, eliminating the need for a second pin. The SN74LVC2G14 inverter, shown in Fig. 13b, is used for this purpose. Additionally, it enhances motor control by filtering noise and providing cleaner signal transitions, improving stability, as illustrated in Fig. 15.

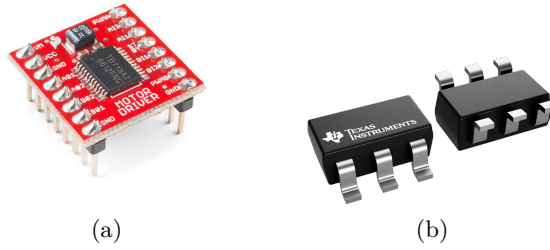


Figure 13: (a) TB6612FNG motor driver [26]; (b) SN74LVC2G14 dual Schmitt-trigger inverter [27].

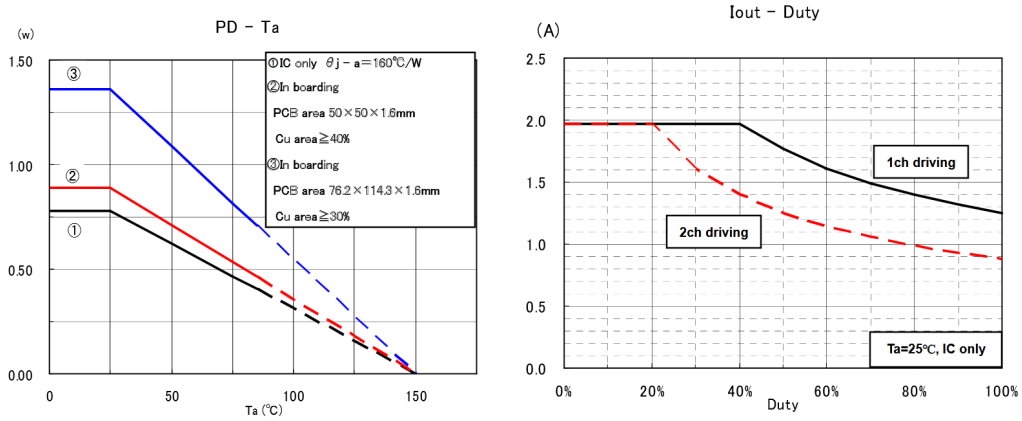


Figure 14: Target characteristics for TB6612FNG [25].

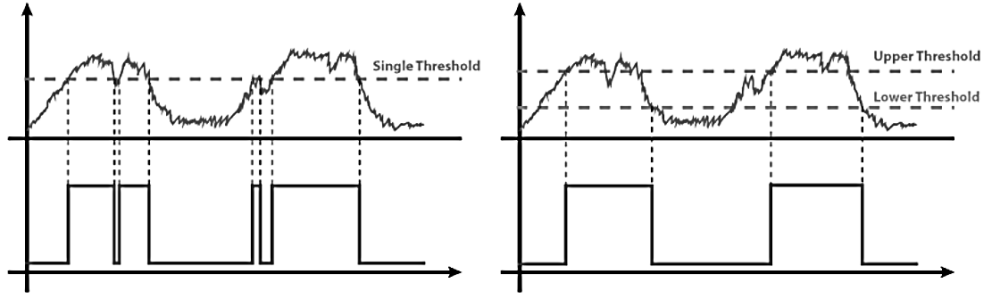


Figure 15: Schmitt trigger output without hysteresis (left) and with hysteresis (right) [28].

When used in conjunction with the TB6612FNG H-Bridge, the SN74LVC2G14 automatically generates the complementary control signals required for motor direction, reducing the number of MCU pins from four to two. As illustrated in Fig. 16, the AIN2 signal is inverted to produce AIN1, and BIN2 is inverted to produce BIN1, eliminating the need for separate MCU outputs for each.

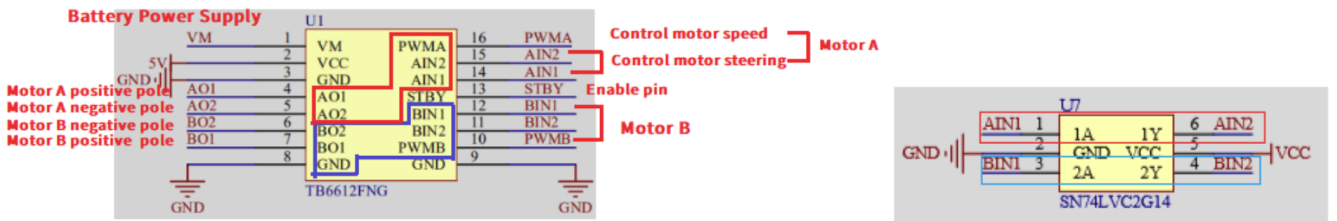


Figure 16: Using SN74LVC2G14 in conjunction with TB6612FNG to reduce MCU pin usage and noise filtering [7].

5.2.6. DC Drive Motors

The drive system employs NHYTEch GA37 520 DC motors (37mm diameter, 12V, 360 RPM) equipped with Hall effect encoders (shown in Fig. 17) [29]. These motors feature a reduction gearbox, which enhances torque output while maintaining controlled rotational speed, making them well-suited for applications requiring precise motion control. The Hall-effect encoders generate quadrature signals, enabling accurate measurement of speed and position.

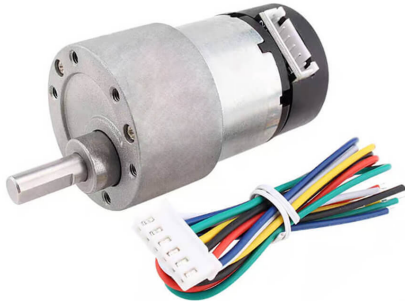


Figure 17: NHYTEch GA37 520 DC motor [29].

5.2.7. Infrared Sensing

The robot is equipped with infrared proximity sensors at the front-left and front-right directions using the Everlight Elec IR Receiver (IRM-56384) and the Infrared LED (IR204C-A), as shown in Fig 18. These sensors detect obstacles by transmitting a modulated infrared signal and detecting its reflection.

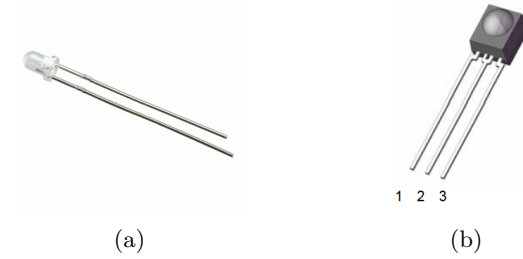


Figure 18: (a) IR204C-A infrared LED [31]; (b) IRM-56384 infrared receiver [32].

5.2.8. Ultrasonic Distance Sensor

The HC-SR04 is an ultrasonic distance sensor, shown in Fig. 19a, used for measuring the distance to an obstacle by sending an ultrasonic pulse and measuring the time it takes for the echo to return. It operates based on the principle of time-of-flight of sound waves, with a known speed of sound in air. The operation of the HC-SR04 ultrasonic sensor is discussed in detail in Section 8.

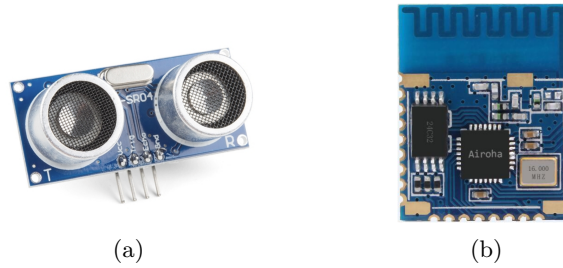


Figure 19: (a) HC-SR04 ultrasonic distance sensor [30]; (b) Bluetooth module [33].

5.2.9. Bluetooth Module Integration

The BT16 Bluetooth UART Module was integrated to provide a robust wireless communication link between the user and the mobile base. The BT16 Bluetooth UART Module is a high-performance component based on the Airoha ABI 602 chipset, compliant with the Bluetooth 4.2 BLE standard. It interfaces with the microcontroller via UART, ensuring efficient bidirectional data transmission. The module supports GATT-based communication through an integrated transparent data transmission service, enabling seamless and reliable Bluetooth connectivity.

A key feature of the BT16 module is its *serial command mode*, which allows direct configuration and control via UART commands. Users can modify essential parameters, such as the UUID, device name, and connection settings, facilitating flexible integration into various applications.

Using this Bluetooth module, we can establish a wireless serial communication channel between a remote device (e.g., a smartphone or computer) and the Arduino Nano’s serial port. This enables real-time transmission and reception of commands, allowing users to send movement instructions to the robot and receive telemetry data such as position, orientation, and sensor readings. This bidirectional communication ensures effective remote control and monitoring, making it suitable for applications requiring dynamic adjustments and feedback.

5.3. TeraRanger Multiflex

The TeraRanger Multiflex is a modular infrared laser Time-of-Flight (ToF) sensor array, as shown in Fig. 3. The system includes eight ToF sensors, a Multiflex Hub, and a circular 3D printed mount [18]. The TeraRanger Multiflex UART interface operates at 3.3 V output levels and is connected to the ESP32-S3-DevKitC-1 using the default UART configuration, as detailed in Table 1.

Table 1: UART Communication Configuration

Parameter	Value
Baud Rate	115200 bit/s
Data Bits	8
Parity	None
Stop Bits	1

6. Sensor Fusion

Complementary filtering, combined with Kalman filtering techniques, is utilised to estimate the state of pitch (or tilt) angle ψ as illustrated in Fig. 32.

6.1. Complementary Filter

The complementary filter is a simple yet effective method for sensor fusion [34] [35] [36]. It blends high-frequency data from the gyroscope with low-frequency data from the accelerometer. The raw accelerometer readings from IMU can be used to estimate tilt angle ψ_{acc} using following equation,¹

$$\psi_{acc} = \arctan \left(\frac{-A_z}{\sqrt{A_x^2 + A_y^2}} \right) \quad (10)$$

Where, A_x , A_y and A_z are the raw acceleration data acquired from the IMU along x , y and z axes, respectively. Similarly, the raw gyroscope readings from the IMU can be used to estimate the angular velocity $\dot{\psi}_{gyro,bias}$,

$$\dot{\psi}_{gyro,bias} = \dot{\psi}_{gyro} - \dot{\psi}_{bias} \quad (11)$$

Where, $\dot{\psi}_{gyro}$ is the raw data from the gyroscope of the IMU and $\dot{\psi}_{bias}$ represents the bias of the gyroscope, which is constant for the IMU used in this project. The value for $\dot{\psi}_{bias}$ is calculated by collecting multiple readings and averaging. If Δt is the sample time and ψ_{prev} is the previous angle value then the estimate tilt angle using gyroscope values can be calculated as following,

$$\psi_{gyro} = \psi_{prev} + \dot{\psi}_{gyro,bias} \cdot \Delta t \quad (12)$$

Then the final estimated pitch angle ψ is calculated as,

$$\psi = \alpha \cdot \psi_{gyro} + (1 - \alpha) \cdot \psi_{acc} \quad (13)$$

Where α is the filter coefficient. The value for filter coefficient through testing was to 0.9 which significantly reduces accelerometer noise while keep convergent time short. The results are shown in 20.

6.2. Extended Kalman Filter

Kalman filter was selected for this self-balancing robot mobile base because of its effectiveness in enhancing sensor fusion for precise state estimation and its capacity to reduce costs by effectively utilising lower-cost sensors [37] [38] [39].

The Kalman filter (KF) provides a more sophisticated approach, estimating the true state of the system by minimizing the mean of the squared error. It involves prediction and update steps, and

¹ $\arctan(y, x) = \begin{cases} \arctan\left(\frac{y}{x}\right) & \text{if } x > 0, \\ \arctan\left(\frac{y}{x}\right) + \pi & \text{if } x < 0 \text{ and } y \geq 0, \\ \arctan\left(\frac{y}{x}\right) - \pi & \text{if } x < 0 \text{ and } y < 0, \\ +\frac{\pi}{2} & \text{if } x = 0 \text{ and } y > 0, \\ -\frac{\pi}{2} & \text{if } x = 0 \text{ and } y < 0, \\ \text{undefined} & \text{if } x = 0 \text{ and } y = 0. \end{cases}$

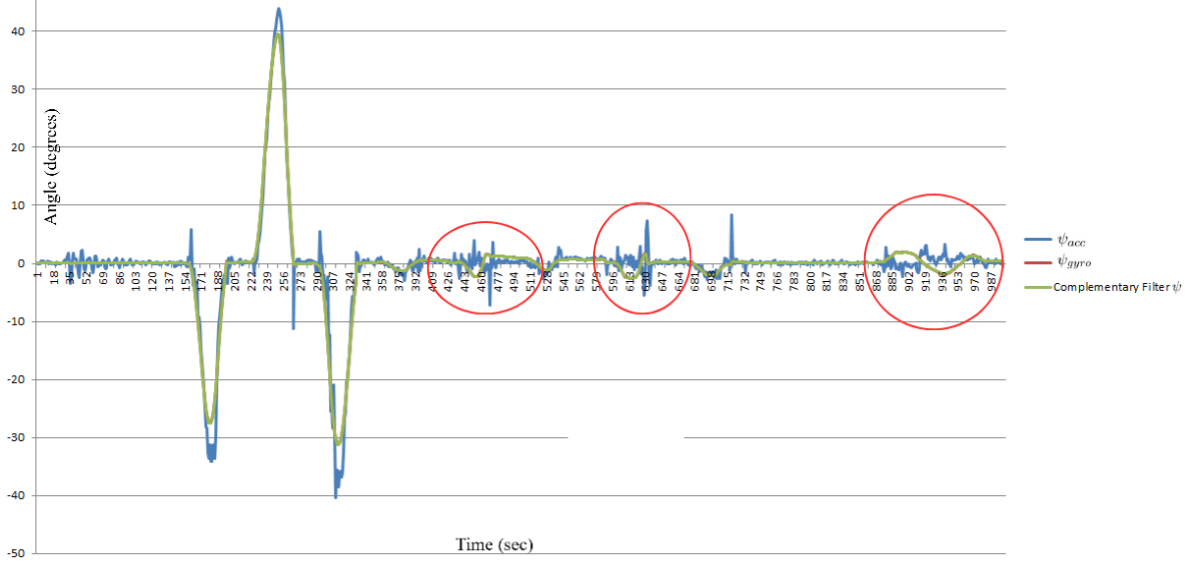


Figure 20: Pitch angle calculated using a complementary filter with $\alpha = 0.9$. It can be observed that the high frequency accelerometer noise (circled in red) are filtered properly. And similarly the low frequency integration errors of the gyroscope (red) are removed over longer interval of time.

assumes that both the process and measurement models are linear. However, many real-world systems, such as robotic motion and sensor fusion, exhibit inherent non-linearities. When linearization is not feasible, the standard KF becomes inaccurate.

To address this limitation, the Extended Kalman Filter (EKF) extends the KF to non-linear systems by employing a first-order Taylor series expansion. This method approximates the non-linear system dynamics and measurement models with locally linearized representations, allowing the filter to be applied in scenarios where exact linearization is not feasible. EKF maintains a Gaussian belief over the state, updating it through a prediction-correction cycle.

6.2.1. Extended Kalman Filter for Estimating Pitch Angle

The estimate of vertical angle ψ can be calculated based on previous angle $\psi_{previous}$ and angular velocity $\dot{\psi}$,

$$\psi = \psi_{previous} + (\dot{\psi} - \dot{\psi}_b) * \Delta t$$

Where the state vector can be defined as $x(t) = \begin{bmatrix} \psi \\ \dot{\psi}_b \end{bmatrix}$. Where $\dot{\psi}_b$ denotes the bias or drift error of the gyroscope. Similarly, the measurement equation for is defined as,

$$y = \dot{\psi}_{gyro}$$

Where, $\dot{\psi}_{gyro}$ is the pitch angular velocity measured by the gyroscope inside IMU. The derivation of equations for the Kalman filter is available in Appendix B.

6.2.2. Predicted State Estimate

From Eq. 73, the discrete current state estimate $\hat{x}_{k|k-1}$ can be written as,

$$\hat{x}_{k|k-1} = G \hat{x}_{k-1|k-1} + H \dot{\psi}_{gyro,k}$$

Where, $\hat{x}_{k-1|k-1}$ is a priori state estimate, and $\dot{\psi}_{gyro,k}$ is the angular velocity measured by the gyroscope at current step k . Also G is the state transition and is equal to $\begin{bmatrix} 1 & -\Delta t \\ 0 & 1 \end{bmatrix}$, and H is the control input and is equal to $\begin{bmatrix} \Delta t \\ 0 \end{bmatrix}$.

Using above equation we can derive following discrete state equation,

$$\psi_{k|k-1} = \psi_{k-1|k-1} + \Delta t(\dot{\psi} - \dot{\psi}_b)_{k-1|k-1}$$

Where, $\psi_{k|k-1}$ and $\psi_{k-1|k-1}$ is a priori and previous state estimate, respectively, of the vertical angle of the robot. And the gyroscope bias $\dot{\psi}_b$ stay constant (e.g. $\dot{\psi}_{b,k|k-1} = \dot{\psi}_{b,k-1|k-1}$).

6.2.3. Predicted Error Covariance

From Eq. 79, the priori measurement covariance matrix $P_{k|k-1}$ was derived to be,

$$P_{k|k-1} = \begin{bmatrix} P_{00} & P_{01} \\ P_{10} & P_{11} \end{bmatrix}_{k|k-1} = \begin{bmatrix} P_{00}^- + [(Q_{\psi} + \Delta t P_{11} - P_{01}^- - P_{10}^-) * \Delta t] & P_{01}^- - (P_{11}^- * \Delta t) \\ P_{10}^- - (P_{11}^- * \Delta t) & P_{11}^- + (Q_{\dot{\psi}_b} * \Delta t) \end{bmatrix}$$

Where the negative sign (e.g. " - ") on top of P_{00}^- , P_{10}^- , P_{11}^- and P_{01}^- represents that these parameters were calculated using previous step $k - 1$ values. And Q_{ψ} is the accelerometer process noise variance and $Q_{\dot{\psi}_b}$ is the process noise variance of gyroscope bias. As suggested in [40], the value of Q_{ψ} can be set to 0.001, while $Q_{\dot{\psi}_b}$ is recommended to be set at 0.003 for most IMUs.

6.2.4. Innovation Residual

From Eq. 78, the innovation residual $\tilde{y}_k$ can be calculated as,

$$\tilde{y}_k = z_k - \psi_{k|k-1}$$

Where, z_k is the measurement made at time step k by combining the values of ψ_{acce} and ψ_{gyro} using Eq. 13.

6.2.5. Innovation Covariance

From Eq. 79, the innovation covariance S_k can be calculated as,

$$S_k = P_{00,k|k-1} + R_{\psi}$$

Where, R_{ψ} is the measurement covariance and is simply equal to the variance of IMU sensor $var(v)$ (see Eq. 72). According to the datasheet for the MPU6050, this variance is approximately 0.003 [24].

6.2.6. Optimal Kalman Gain

From Eq. 81, the optimal Kalman gain K_k can be calculated as,

$$K_k = \begin{bmatrix} K_0 \\ K_1 \end{bmatrix} = \begin{bmatrix} \frac{P_{00}}{S_k} \\ \frac{P_{10}}{S_k} \end{bmatrix}$$

6.2.7. Corrected State Estimate

From Eq. 82, the updated (or posteriori) state estimate $\hat{x}_{k|k}$ can be calculated as,

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k \tilde{y}_k$$

6.2.8. Corrected Estimate Covariance

From Eq. 83, the posteriori measurement covariance matrix $P_{k|k}$,

$$P_{k|k} = \begin{bmatrix} P_{00} - K_0 \cdot P_{00} & P_{01} - K_0 \cdot P_{01} \\ P_{10} - K_1 \cdot P_{00} & P_{11} - K_1 \cdot P_{01} \end{bmatrix} \quad (14)$$

6.3. Software Implementation of EKF

For the implementation of the EKF, we utilized the Kalman filter library developed by Kristian Lauszus [40]. This library was modified in accordance with the GNU General Public License to meet the specific requirements of our project. The pseudocode for the filtering process is presented in Algorithm 2.

Algorithm 2 Kalman Filter Angle Estimation

```
1: procedure ESTIMATEANGLE(measuredAngle, measuredGyro)
  Prediction Step:
2:    $angle \leftarrow angle + (measuredGyro - bias) \cdot \Delta t$       ▷ Update angle based on gyroscope data
3:    $y \leftarrow measuredAngle - angle$                           ▷ Calculate measurement residual
  Covariance Matrix Time Update:
4:    $P_{00} \leftarrow P_{00} + (P_{11} \cdot \Delta t + Q_{angle} - P_{01} - P_{10}) \cdot \Delta t$       ▷ Update covariance for angle
5:    $P_{01} \leftarrow P_{01} - P_{11} \cdot \Delta t$                       ▷ Update cross-covariance
6:    $P_{10} \leftarrow P_{10} - P_{11} \cdot \Delta t$                       ▷ Update cross-covariance
7:    $P_{11} \leftarrow P_{11} + Q_{gyro} \cdot \Delta t$                   ▷ Update covariance for gyro
  Compute Kalman Gain:
8:    $S \leftarrow R_{angle} + P_{00}$                                 ▷ Calculate innovation covariance
9:    $K_0 \leftarrow \frac{P_{00}}{S}$                                     ▷ Calculate Kalman gain for angle
10:   $K_1 \leftarrow \frac{P_{10}}{S}$                                     ▷ Calculate Kalman gain for bias
  Correct the Angle and Bias:
11:   $angle \leftarrow angle + K_0 \cdot y$                           ▷ Update angle estimate
12:   $bias \leftarrow bias + K_1 \cdot y$                             ▷ Update bias estimate
  Update the Covariance Matrix:
13:   $P_{00} \leftarrow P_{00} - K_0 \cdot P_{00}$                     ▷ Update covariance for angle
14:   $P_{01} \leftarrow P_{01} - K_0 \cdot P_{01}$                     ▷ Update cross-covariance
15:   $P_{10} \leftarrow P_{10} - K_1 \cdot P_{00}$                     ▷ Update cross-covariance
16:   $P_{11} \leftarrow P_{11} - K_1 \cdot P_{01}$                     ▷ Update covariance for gyro
17:  return angle                                              ▷ Return the estimated angle
18: end procedure
```

7. Motor Control

The system employs a Proportional-Integral-Derivative (PID) control loop to achieve real-time balance and motion control of the robot. This control architecture ensures stability by continuously monitoring and adjusting the robot's state using feedback from multiple sensors [41]. Key inputs to the control loop include: the robot's pitch angle ψ (as illustrated in Fig. 32), gyroscope data, and motor encoder values, which provide critical information on orientation, angular velocities, and position.

- **Pitch angle:** Provides information about the robot's tilt relative to the vertical y -axis.
- **Gyroscope data:** Supplies angular velocity measurements for dynamic stabilization.
- **Motor encoder values:** Tracks relative wheel position and velocity for precise motion control.

The control algorithm computes outputs for pitch, yaw, and position at regular intervals, generating pulse-width-modulation (PWM) signals to adjust motor speeds via the motor driver. A control block diagram for pitch angle ψ PID controller is shown in Fig. 21.

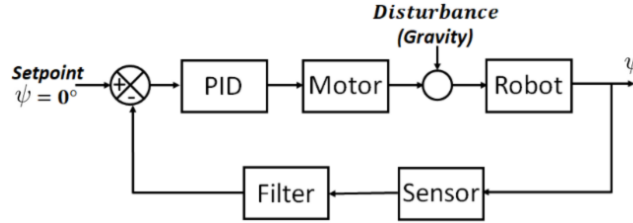


Figure 21: Block diagram for pitch angle ψ control.

Similarly, a PID controller is utilized to manage both the yaw angle and distance. To prioritize system stability, the pitch angle is updated at a higher frequency (e.g., every control cycle), while the yaw angle and position control outputs are updated less frequently (e.g., every eighth cycle). This strategy optimizes resource utilization while ensuring robust performance.

7.1. PID Controller

The PID controller generates a control signal $u(t)$ based on the error signal $e(t)$, which is the difference between the desired set-point and the measured process variable. The control signal is computed as follows,

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d}{dt} e(t) \quad (15)$$

Where:

- $u(t)$: Control signal applied to the system.
- $e(t)$: Error signal, representing the deviation from the setpoint.
- K_p : Proportional gain, which responds to the current error.
- K_i : Integral gain, which addresses accumulated past errors.

- K_d : Derivative gain, which predicts future error trends based on the rate of change.

For digital implementation, the continuous-time PID equation is discretized to suit microcontroller-based systems. The discrete-time PID control signal $u[n]$ is calculated as:

$$u[n] = K_p e[n] + K_i T_s \sum_{k=0}^n e[k] + K_d \frac{e[n] - e[n-1]}{T_s} \quad (16)$$

Where, T_s is sampling period, representing the time interval between consecutive control updates, $e[n]$ error signal at the n -th sampling instant and $u[n]$ control signal at the n -th sampling instant. This formulation ensures compatibility with real-time embedded systems while maintaining control precision.

7.2. PID Tuning Methodology

The Ziegler-Nichols method is a widely used and systematic approach for tuning PID controllers. It provides a reliable framework for determining initial controller parameters, which can then be fine-tuned for optimal performance. The method involves inducing controlled oscillations in the system to identify critical parameters, which are then used to calculate the proportional, integral, and derivative gains.

7.2.1. Ziegler-Nichols Method

The Ziegler-Nichols tuning procedure consists of the following steps:

1. **Initialize Parameters:** Set the integral gain K_i and derivative gain K_d to zero, leaving only the proportional gain K_p active.
2. **Induce Oscillations:** Gradually increase K_p until the system exhibits sustained oscillations. At this point, the system is at the threshold of stability, and the proportional gain is referred to as the ultimate gain K_u . The period of these oscillations is denoted as T_u .
3. **Record Critical Values:** Note the values of K_u and T_u , as they are essential for calculating the final PID parameters.
4. **Calculate PID Gains:** Using the recorded values, compute the PID parameters as follows:

$$\begin{aligned} K_p &= 0.6K_u \\ T_i &= 0.5T_u \\ T_d &= 0.125T_u \end{aligned} \quad (17)$$

Here, T_i and T_d represent the integral and derivative time constants, respectively. These values are then used to determine the integral and derivative gains:

$$K_i = \frac{K_p}{T_i} \quad \text{and} \quad K_d = K_p \cdot T_d. \quad (18)$$

This method provides a robust starting point for achieving stable control. However, it is important to note that the Ziegler-Nichols method may require additional fine-tuning to account for system-specific

dynamics and performance requirements. The calculated parameters serve as an initial baseline, which can be further optimized through iterative testing and adjustment.

7.2.2. Practical Tuning Guidelines

In addition to the Ziegler-Nichols method, practical tuning guidelines were applied to refine the controller performance:

- Start with a small proportional gain K_p (e.g. $K_p = 10$) to avoid instability.
- Introduce the derivative term K_d to dampen oscillations, typically setting $K_d = 0.1K_p$.
- Fine-tune K_p , K_i , and K_d iteratively to achieve optimal stability and responsiveness.

7.3. Multi-rate PID Control

To optimize system performance and resource utilization, the control loop employs a Multi-rate PID strategy:

- **Pitch Control:** Updated at the highest frequency (e.g., every control cycle) to ensure rapid response to changes in the robot's tilt and maintain balance.
- **Yaw and Position Control:** Updated at a lower frequency (e.g., every 8th cycle) to reduce computational load while still providing adequate motion control.

This approach prioritizes critical tasks (e.g., maintaining balance) while efficiently managing system resources, ensuring robust and stable operation. A simplified flow diagram for a Multi-rate PID controller is shown in Fig. 22.

7.3.1. Pitch Control

The pitch control loop ensures the robot maintains its upright position. The primary objective of the pitch controller is to minimize the deviation of the robot's pitch angle ψ from a set-point. The pitch control output is calculated using the PD algorithm, where the error is the difference between the current pitch angle and the desired pitch angle. However, the integration component of the PD controller is unnecessary due to the system's inherent instability. Instead, a proportional and derivative (PD) control strategy is more appropriate. The equation for the PD controller is as follows,

$$\tau_{\psi,pd} = K_{p\psi}(\psi_{desired} - \psi_{measured}) + K_{d\psi} \frac{d}{dt}(\psi_{desired} - \psi_{measured}) \quad (19)$$

Where, $\tau_{\psi,pd}$ is the pitch control output, $\psi_{desired}$ is the desired pitch angle and $\psi_{measured}$ is the estimated pitch angle obtained using the Extended Kalman Filter, as calculated in Eq. 82.

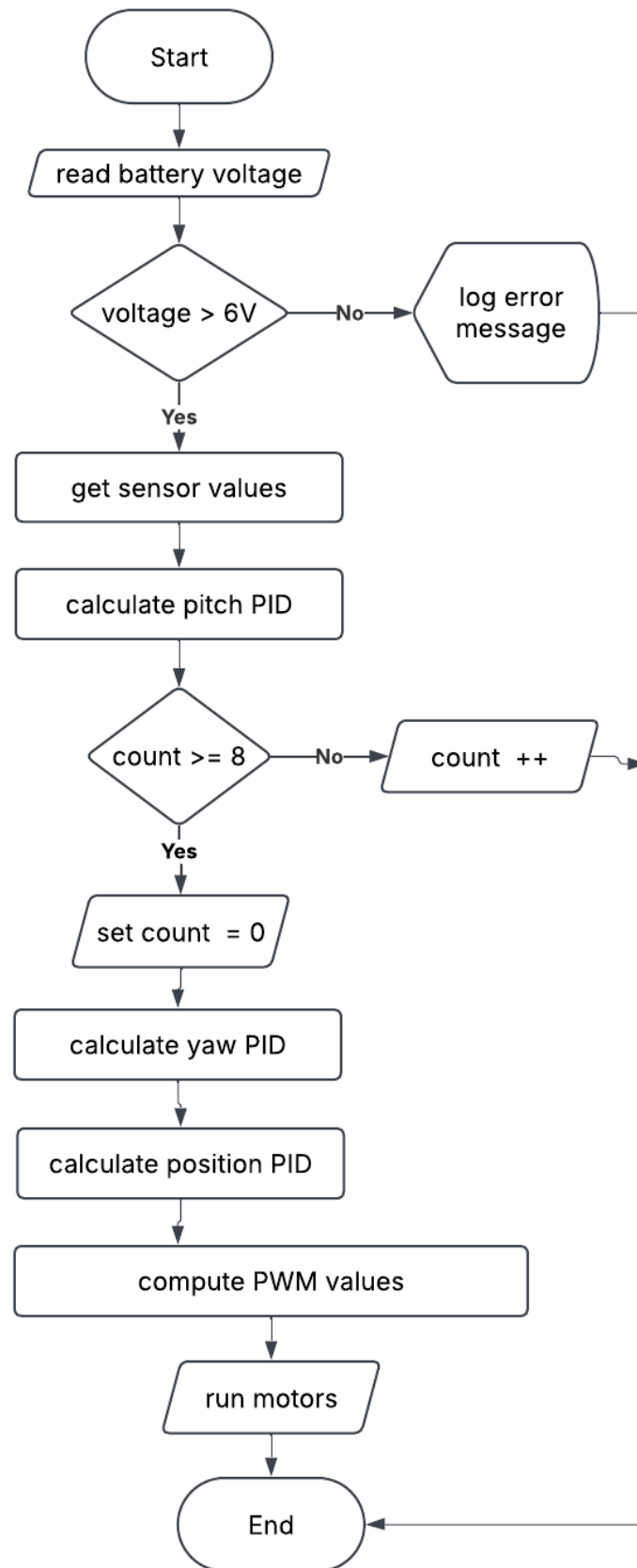


Figure 22: A simplified flow diagram of the multi-rate PID control loop used.

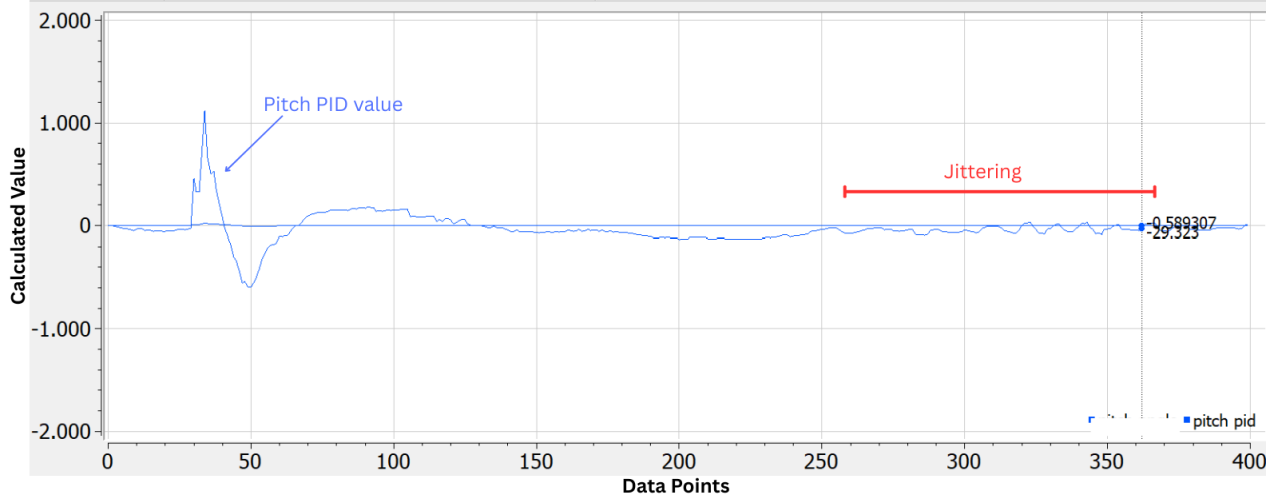


Figure 23: Output values calculated by the pitch angle PD controller at $K_{p\psi} = 55$ and $K_{d\psi} = 1.25$. The data was recorded at baud rate of 115200.

The output values calculated by the pitch angle PD controller, shown in Fig. 23, indicates that system quickly reaches the desired vertical position. However, once stability is achieved, the system begins to exhibit jitter—characterised by small, rapid fluctuations in the control output—particularly evident in the PD output between data points 300 and 400 in Fig. 23. This jitter is a result of the high sensitivity of the derivative gain ($K_{d\psi}$) to fluctuations in the gyroscope readings ($\dot{\psi}$). While reducing $K_{d\psi}$ mitigates the jitter, it also diminishes the system’s ability to dampen oscillations, leading to excessive overshoot and potential instability, as shown in Fig. 24.

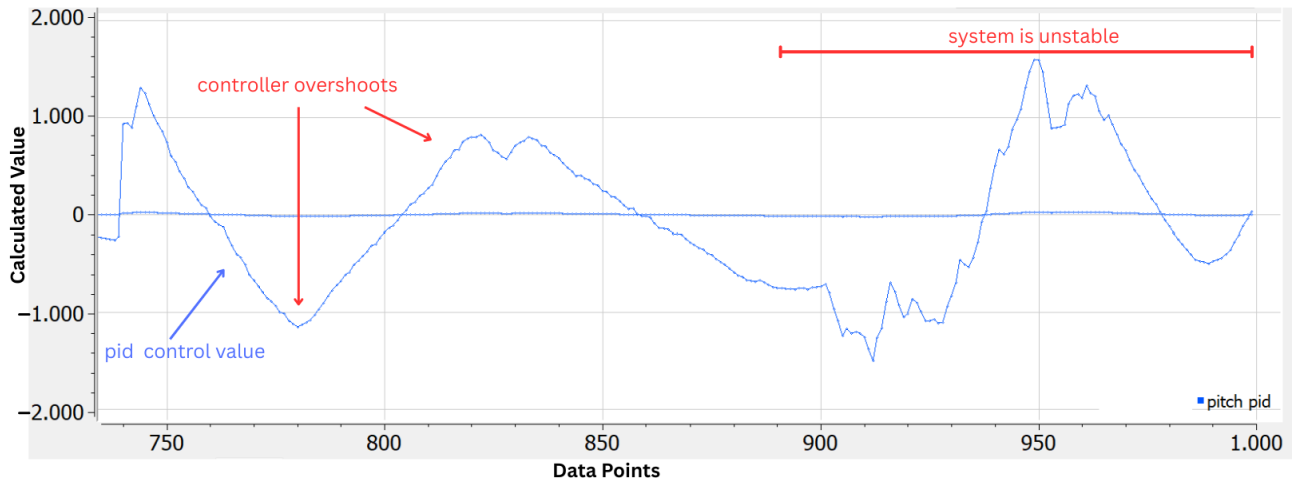


Figure 24: Output values calculated by the pitch angle PD controller at $K_{p\psi} = 55$ and $K_{d\psi} = 0.75$. Data recorded at baud rate of 115200.

It can be observed from Fig. 24 that the system overshoots primarily due to low value of K_d . This insufficient derivative gain leads to excessive oscillations, ultimately resulting in instability, as evidenced by the data points from 900 onwards. To address this issue, an adaptive derivative gain approach is implemented, where $K_{d\psi}$ is adjusted dynamically based on the pitch angle of the robot. When the absolute pitch angle ψ exceeds a predefined threshold, a higher derivative gain is used to provide stronger correction, while a lower gain is applied for small angles to minimize jitter. The

algorithm used for implementing adaptive derivative gain is as follows,

Algorithm 3 Adaptive Derivative Gain Logic

```

1: if pitch_angle > 8 degrees then                                ▷ predefined small angle threshold
2:   kd ← 1.25                                                         ▷ for large angles
3: else
4:   kd ← 0.75                                                         ▷ for small angles
5: end if

```

This adaptive control strategy effectively stabilises the system, resulting in reduced oscillations and the elimination of jitter, as shown Fig. 25.

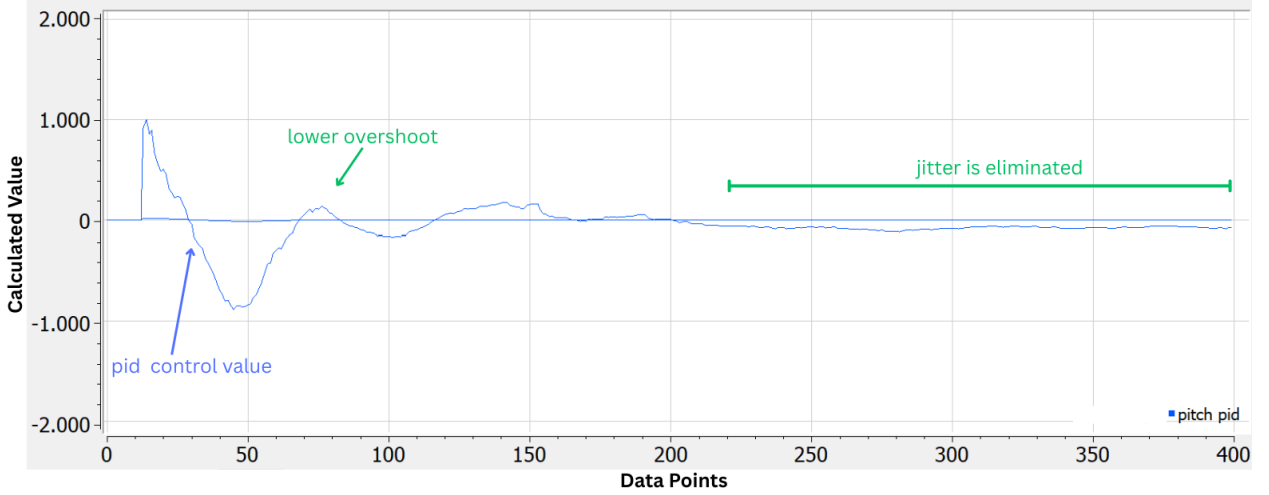


Figure 25: Output values calculated by the pitch angle PD controller with an adaptive derivative gain at $K_{p\psi} = 55$ and $K_{d\psi} = 0.75 \sim 1.25$. The data was recorded at baud rate of 115200.

7.3.2. Yaw Control

Yaw control is responsible for controlling the robot's rotational movement around its vertical y -axis shown in Fig. 32. After testing, it was determined that the steady-state error is minimal, indicating that a PD controller is adequate for effective control. The yaw PD controller computes the control output based on the robot's angular velocity, which is measured by the gyroscope along the y -axis.

$$\tau_{\phi, pd} = K_{p\phi}(\phi_{desired} - \phi) + K_{d\phi} \frac{d}{dt}(\phi_{desired} - \phi) \quad (20)$$

Where, $\tau_{\phi, pd}$ is the yaw control output, $\phi_{desired}$ is the desired yaw angle, and ϕ is the estimate of angle of rotation about y -axis. The yaw angle ϕ can be calculated based on previous yaw angle $\phi_{previous}$ and angular velocity $\dot{\phi}_{gyro}$ measured by the gyroscope using following equation,

$$\phi = \phi_{previous} + (\dot{\phi}_{gyro} - \dot{\phi}_b) * \Delta t$$

Where $\dot{\phi}_b$ denotes the bias or drift error of the gyroscope. The yaw angle is measured relative to the initial yaw angle when the system is initiated, indicating that it is not aligned with the x -axis shown in Fig. 32.

The yaw control adjusts the motor speeds to achieve the desired angle, ensuring the robot maintains a stable heading. Fig. 26 illustrates the system's response with $K_{p\phi} = 2.5$ and $K_{d\phi} = 0.5$ when transitioning from an initial angle of -20 deg to final angle of $+20 \text{ deg}$.

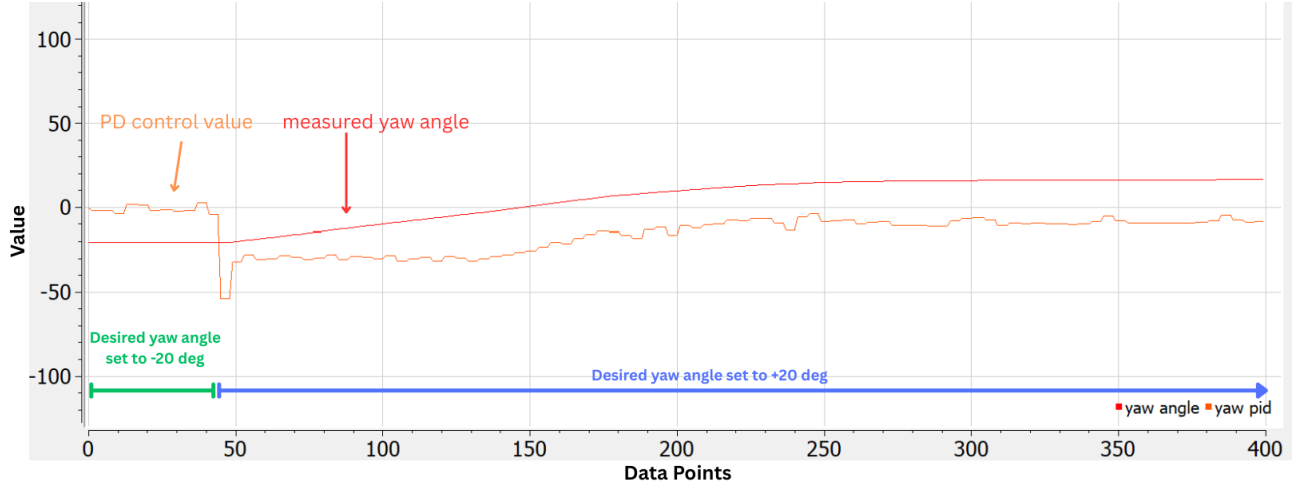


Figure 26: Yaw angle control system response at $K_p = 2.5$ and $K_d = 0.5$ for angle varying from -20 deg to $+20 \text{ deg}$. The data was recorded at baud rate of 115200.

As shown in the Fig. 26, the robot moves to the desired yaw angle changes smoothly without any overshoot.

7.3.3. Position Control

Due to integration errors associated with estimating distance from acceleration values obtained from the MPU6050, the encoder values from the Hall-effect encoders in the DC drive motors were utilized for distance estimation along the x -axis shown in Fig. 32. The raw data obtained from encoders is processed using a Weighted Moving Average (WMA) filter to estimate the distance.

The encoder pulses are counted using an Interrupt Service Routine (ISR)—a specialized piece of code that executes in response to an interrupt signal, allowing for efficient handling of asynchronous events—triggered by changes on the encoder signal pins, as shown in Algorithm 4. After each control cycle the encoder counts are accumulated while taking into account the direction of the motors using *UpdateDistance* function as shown in Algorithm 5. To further enhance the accuracy of position estimation, a Weighted Moving Average (WMA) filter is applied to reduce noise, as shown in Algorithm 5. The filtered encoder pulses ω_{filtered} is calculated using the formula:

$$\omega_{\text{filtered}} = \alpha \cdot \omega_{\text{prev}} + (1 - \alpha) \cdot \omega_{\text{raw}} \quad (21)$$

Where, ω_{prev} is the previously filtered pulses, ω_{raw} is the newly computed pulses from encoder readings and α is the weighting factor. In this project $\alpha = 0.7$ was selected after testing. The implementation of the WMA filter is also shown in Algorithm 5, where constant the conversion factor for translating encoder pulses into distance was determined to be approximately 0.03125 centimetres per encoder pulse.

Position control is implemented to ensure the robot moves smoothly and accurately to the desired location. The encoder feedback from the left and right wheels is used to calculate the robot's

Algorithm 4 Interrupt Handlers for Encoder Counts

```

1: Attach interrupt on edge change of LEFT_ENCODER_PIN to LEFTENCODERISR
2: Attach interrupt on edge change of RIGHT_ENCODER_PIN to RIGHTENCODERISR
3: procedure LEFTENCODERISR
4:    $L\_Count \leftarrow L\_Count + 1$  ▷ Increment left encoder count
5: end procedure
6: procedure RIGHTENCODERISR
7:    $R\_Count \leftarrow R\_Count + 1$  ▷ Increment right encoder count
8: end procedure

```

Algorithm 5 Update Total Distance

```

1: procedure UPDATEDISTANCE
2:   if Left motor moving forward then
3:      $L\_Total \leftarrow L\_Total + L\_Count$ 
4:   else
5:      $L\_Total \leftarrow L\_Total - L\_Count$ 
6:   end if
7:   if Right motor moving forward then
8:      $R\_Total \leftarrow R\_Total + R\_Count$ 
9:   else
10:     $R\_Total \leftarrow R\_Total - R\_Count$ 
11:   end if
12:    $L\_Total_{filtered} \leftarrow (L\_Total_{filtered,old} \times 0.7) + (L\_Total \times 0.3)$ 
13:    $L\_Total_{filtered,old} \leftarrow L\_Total_{filtered}$ 
14:    $L\_Position \leftarrow L\_Position + L\_Total_{filtered} \times CM\_PER\_ENCODER\_PULSES$ 
15:    $R\_Total_{filtered} \leftarrow (R\_Total_{filtered,old} \times 0.7) + (R\_Total \times 0.3)$ 
16:    $R\_Total_{filtered,old} \leftarrow R\_Total_{filtered}$ 
17:    $R\_Position \leftarrow R\_Position + R\_Total_{filtered} \times CM\_PER\_ENCODER\_PULSES$ 
18: end procedure

```

displacement and speed. The position PD controller adjusts the motor speeds to minimize the error in position. As shown in the Fig. 27, the robot moves to the desired position smoothly and without any overshoot.

$$\tau_{x,pd} = K_{px}(x_{desired} - x_{measured}) + K_{dx} \frac{d}{dt}(x_{desired} - x_{measured}) \quad (22)$$

Where, $\tau_{x,pd}$ is the position control output, $x_{desired}$ is the desired position and $x_{measured}$ is the measured position. Fig. 27 presents the system's response when $K_{px} = 0.26$ and $K_{dx} = 20$ when moving from relative distance of -600 mm to $+300 \text{ mm}$. As shown in the Fig. 27, the robot moves to the desired position smoothly and without any overshoot.

7.3.4. Combining Control Outputs

The final motor control is achieved by combining the outputs from all three PD controllers. The outputs from the pitch, yaw, and position PD controllers are used to calculate the motor speeds, which determine the robot's motion. Specifically, the following equations are used to compute the

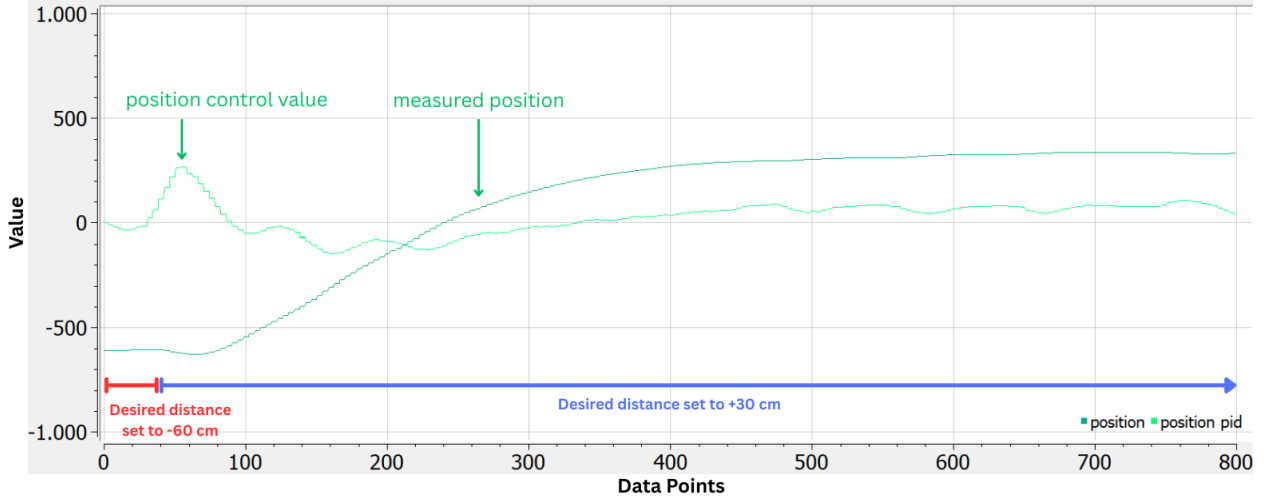


Figure 27: Position control system response with $K_{px} = 0.26$ and $K_{dx} = 20$ for a displacement of -600 mm to $+300\text{ mm}$. Data recorded at baud rate of 115200.

Pulse-Width-Modulation (PWM) values for the left motor $\tau_{left,motor}$ and right motor $\tau_{right,motor}$,

$$\tau_{left,motor} = \tau_{\theta,pd} - \tau_{\phi,pd} - \tau_{x,pd} \quad (23)$$

$$\tau_{right,motor} = \tau_{\theta,pd} + \tau_{\phi,pd} - \tau_{x,pd} \quad (24)$$

The PWM values $+255$ represents maximum forward speed, -255 represents maximum reverse, and 0 represents stationary state. The PWM values are constrained between -255 and $+255$ because a 8-bit timer is used by Arduino Nano to generate the PWM signal [21]. The computed PWM are transmitted to the motor drivers, which adjust the robot's movement and balance in real time. This closed-loop control mechanism ensures precise and stable operation by continuously refining motor outputs based on sensor feedback. A simplified diagram of the motor control loop is illustrated in Fig. 28.

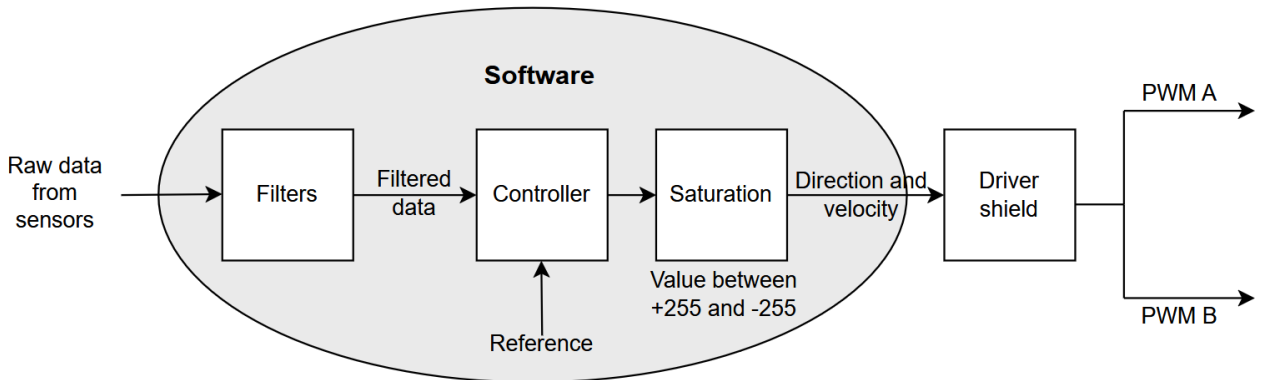


Figure 28: A simplified diagram of the motor control loop [35].

8. Front Obstacle Detection

To robustly detect obstacles in its forward path, the robot employs a combination of an ultrasonic distance-measuring sensor and two infrared (IR) proximity sensors.

8.1. Operation of Ultrasonic sensor

The HC-SR04 ultrasonic module used in this project comprises two key components: a transmitter that emits high-frequency ultrasonic pulses and a receiver that detects the reflected pulses after they bounce off an obstacle. The distance to the obstacle is calculated by measuring the time delay between the transmission of the ultrasonic pulse and its reception. The HC-SR04 ultrasonic has a measuring range of 2cm – 400 cm (1" - 13ft) within a 30 degree cone [30]. The formula used to compute the distance is as follows:

$$d = \frac{t \times v}{2} \quad (25)$$

Where d is the measured distance to the obstacle, t is the time delay between transmission and reception, and v is the speed of sound in air, which is approximately 340.29 m/s at room temperature (e.g. 20°C). Based on the datasheet [30], an operating frequency of 20 Hz is selected for distance measurements.

8.1.1. Sound Signal Transmission

The HC-SR04 requires control signals to be sent from a microcontroller, A short trigger pulse (at least 10 μ s) is sent to the TRIG pin. The following algorithm initiates distance measurement using the HC-SR04 ultrasonic sensor by transmitting a 20Hz acoustic signal, which corresponds to a measurement interval of 50 ms. This means that the sensor can take a new distance measurement every 50 ms. The system periodically initiates a distance measurement by sending a trigger pulse to the ultrasonic sensor, as shown in Algorithm 6.

Algorithm 6 Start Ultrasonic Distance Measurement

```

1: procedure STARTULTRASONICMEASUREMENT
2:   if current time – previousTriggerTime > TRIGGER_DELAY then
3:     Set measurementFlag  $\leftarrow$  SEND
4:     previousTriggerTime  $\leftarrow$  current time
5:     Attach interrupt on rising edge of ECHO_PIN to HANDLEULTRASONICINTERRUPT
6:     Set TRIG_PIN to LOW
7:     Set TRIG_PIN to HIGH
8:     Wait for 10  $\mu$  s
9:     Set TRIG_PIN back to LOW
10:  end if
11: end procedure

```

Algorithm Explanation:

- The function `StartUltrasonicMeasurement` checks if the time elapsed since the last distance measurement exceeds the defined `TRIGGER_DELAY` of 50 ms. This ensures that measurements are taken at regular intervals, preventing overlap and ensuring accurate readings.

- A global flag `measurementFlag` is set to `SEND`, indicating that the trigger pulse is sent.
- Then attaches an interrupt to the `ECHO_PIN` pin, allowing the microcontroller to detect when the echo signal is received.
- The `TRIG_PIN` pin is first set to `LOW` to prepare for the trigger pulse, then set to `HIGH` for 10 μ s to send the trigger pulse, and finally set back to `LOW` to reset the trigger.

This setup enables precise distance measurement by capturing the time delay between sending the ultrasonic pulse and receiving the echo, allowing for accurate distance calculations.

8.1.2. Sound Signal Reception

The HC-SR04 ultrasonic sensor responds to the transmitted acoustic signal by generating a high signal on the `ECHO` pin. The duration of this high signal corresponds to the time taken for the sound waves to travel to an obstacle and back. To accurately measure this duration and calculate the distance, an Interrupt Service Routine (ISR) is employed. The Algorithm 7 outlines the steps taken in the ISR to measure the distance using the received acoustic signal,

Algorithm 7 Ultrasonic Echo Interrupt Handler

```

1: procedure HANDLEULTRASONICINTERRUPT
2:   if usonicMeasureFlag = SEND then
3:     usonicMeasurePrevTime  $\leftarrow$  current time ▷ Rising edge capture
4:     Reconfig. interrupt on falling edge of ECHO_PIN to HANDLEULTRASONICINTERRUPT
5:     usonicMeasureFlag  $\leftarrow$  RECEIVED
6:   else if usonicMeasureFlag = RECEIVED then
7:      $\Delta t \leftarrow$  (current time – usonicMeasurePrevTime)
8:     usonicDistanceValue  $\leftarrow$  ( $\Delta t \times 0.34029$ ) ▷ Convert duration to cm
9:     usonicMeasureFlag  $\leftarrow$  IDLE ▷ Reset state machine
10:  end if
11: end procedure

```

Algorithm Explanation:

1. Rising Edge Detection:

- When the `ECHO` signal transitions to a high state (`RISING` edge), the current timestamp is recorded as `usonicMeasurePrevTime`.
- The interrupt is then reconfigured to detect the falling edge of the `ECHO` signal, which indicates the end of the echo.
- The flag `usonicMeasureFlag` is updated to `RECEIVED` to indicate that the echo signal is being processed.

2. Falling Edge Detection:

- Upon detecting the falling edge, the elapsed time (Δt) is calculated as the difference between the current time and the previously recorded timestamp.

- The distance to the obstacle is then calculated using the formula:

$$d = \frac{\Delta t \times v}{2} \quad (26)$$

Where v is the speed of sound in air, approximately 340.29 m/s.

- Finally, the flag `usonicMeasureFlag` is reset to `IDLE` to prepare the system for the next measurement cycle.

In summary, the ISR efficiently captures the duration of the echo signal by detecting both the rising and falling edges of the `ECHO` pin. This allows for accurate distance calculations based on the time taken for the sound waves to travel to the obstacle and back, ensuring reliable performance of the ultrasonic measurement system.

8.2. Infrared Sensing

Infrared (IR) sensors are employed for proximity-based obstacle detection. The system periodically emits modulated infrared pulses and processes the received reflections to infer the presence of nearby obstacles.

8.2.1. IR Signal Emission

The IR204C-A infrared LED generates a 38 kHz square wave signal by toggling the output pin at 18 μ s intervals, comprising 9 μ s LOW and 9 μ s HIGH phases. This emitted signal is detected by the IR receiver IRM-56384, which is tuned to the frequency of 38 kHz [32]. Algorithm 8 illustrates the signal generation.

Algorithm 8 Transmit 38 kHz IR Pulse

```

1: procedure SEND38KHzPULSE(ir_pin)
2:   for  $i \leftarrow 1$  to 39 do
3:     Set digital output ir_pin to LOW
4:     Wait for 9  $\mu$ s
5:     Set digital output ir_pin to HIGH
6:     Wait for 9  $\mu$ s
7:   end for
8: end procedure

```

8.2.2. IR Signal Reception

The IR receiver IRM-56384 detect modulated infrared signals transmitted by the IR204C-A infrared LED. The receiving mechanism relies on edge-triggered interrupts to accurately count the number of incoming IR pulses. Each IR receiver (left and right) uses an interrupt service routine (ISR) to toggle between detecting rising and falling edges of the IR signal. On the rising edge, a flag is set to indicate that a new pulse may have begun, while on the falling edge, the pulse is confirmed and counted. Algorithm 9 summarizes the interrupt-driven pulse counting strategy used by the IR receivers.

Algorithm 9 IR Signal Pulse Counting Using Edge Detection

```
1: procedure IRLEFTRECEIVE
2:   if irLeftReceiveFlag = false then
3:     irLeftReceiveFlag  $\leftarrow$  true
4:     Enable interrupt on RISING edge of LEFT_RECEIVE_PIN
5:   else
6:     Increment irLeftPulseCount
7:     irLeftReceiveFlag  $\leftarrow$  false
8:     Enable interrupt on FALLING edge of LEFT_RECEIVE_PIN
9:   end if
10: end procedure
11: procedure IRRIGHTRECEIVE
12:   if irRightReceiveFlag = false then
13:     irRightReceiveFlag  $\leftarrow$  true
14:     Enable interrupt on RISING edge of RIGHT_RECEIVE_PIN
15:   else
16:     Increment irRightPulseCount
17:     irRightReceiveFlag  $\leftarrow$  false
18:     Enable interrupt on FALLING edge of RIGHT_RECEIVE_PIN
19:   end if
20: end procedure
```

Algorithm Explanation:

- Each IR receiver function operates as an interrupt service routine triggered by a signal edge.
- On detecting a rising edge, the corresponding flag is set and the interrupt is immediately reconfigured to detect a falling edge.
- On the falling edge, a full pulse is considered received, so the pulse counter is incremented. The flag is cleared, and the interrupt is again set to detect a rising edge.
- This alternating edge detection mechanism ensures robust counting of complete IR pulses and minimizes missed detections caused by noise or signal jitter.

This interrupt-based approach allows the system to efficiently track IR signal activity with minimal processing overhead, enabling real-time response to environmental cues and communication signals.

8.2.3. Binary Filter:

To ensure robust and noise-resilient obstacle detection, a binary filtering technique is applied to the IR pulse counts. The system employs a sliding window mechanism that evaluates whether an obstacle is present by analyzing a sequence of binary decisions over time. This approach reduces false positives caused by transient disturbances or environmental noise.

At regular intervals, the IR sensors are activated to transmit a modulated 38 kHz signal. The number of pulses received is evaluated, and if the pulse count exceeds a predefined threshold, the result is considered as a binary “1” (obstacle detected); otherwise, a binary “0” is recorded. These binary decisions are then passed into a circular sliding window, which maintains a running count of obstacle-positive entries. If the number of positive entries within the window exceeds a defined

threshold, the presence of an obstacle is confirmed. The complete algorithm for this filtering method is outlined in Algorithm 10.

Algorithm 10 Binary Filter for Obstacle Detection

```
1: procedure CHECKIROBSTACLE
2:   if current time – irSendTime > IR_PULSE_DELAY_MS then
3:     Send 38kHz IR pulse from IR_SEND_PIN
4:     irSendTime ← current time
5:   end if
6:   PROCESSLEFTIRSENSOR
7:   PROCESSRIGHTIRSENSOR
8: end procedure
9: procedure PROCESSLEFTIRSENSOR
10:  if current time – irLeftCountTime > IR_COUNT_DELAY_MS then
11:    obstacleDetected ← (irLeftPulseCount ≥ 3)
12:    UPDATESLIDINGWINDOW(obstacleDetected, irLeftHistory, irLeftIndex,
irLeftRunningCount)
13:    irLeftIsObstacle ← (irLeftRunningCount ≥ 5)
14:    Reset irLeftPulseCount and update irLeftCountTime
15:  end if
16: end procedure
17: procedure PROCESSRIGHTIRSENSOR
18:  if current time – irRightCountTime > IR_COUNT_DELAY_MS then
19:    obstacleDetected ← (irRightPulseCount ≥ 3)
20:    UPDATESLIDINGWINDOW(obstacleDetected, irRightHistory, irRightIndex,
irRightRunningCount)
21:    irRightIsObstacle ← (irRightRunningCount ≥ 5)
22:    Reset irRightPulseCount and update irRightCountTime
23:  end if
24: end procedure
25: procedure UPDATESLIDINGWINDOW(value, history, index, runningCount)
26:  runningCount ← runningCount – history[index] + value
27:  history[index] ← value
28:  index ← (index + 1) mod SLIDING_WINDOW_SIZE
29: end procedure
```

Algorithm Explanation:

- The function `CheckIRObstacle` regularly sends IR pulses and then calls individual sensor processing functions for left and right IR receivers.
- In `ProcessLeftIRSensor` and `ProcessRightIRSensor`, the system checks if the IR pulse count over a sampling period exceeds a threshold (e.g., 3 pulses). If so, it considers that a binary detection of an obstacle has occurred.
- A binary value (1 for obstacle, 0 otherwise) is inserted into a fixed-size sliding window using the `UpdateSlidingWindow` function.
- The running count in the window is updated in constant time by subtracting the value being replaced and adding the new value.

- If enough detections (e.g., 5) occur within the window, an obstacle is confirmed on the respective side.

This binary filtering method offers computational efficiency and robust noise immunity, making it well-suited for real-time embedded systems where sensor signals may fluctuate due to ambient interference or hardware limitations.

8.3. Obstacle Decision Logic

To reliably determine the presence of a front obstacle, the system fuses measurements from the HC-SR04 ultrasonic sensor and the binary-filtered outputs of both IR sensors. An obstacle is confirmed if either:

- The ultrasonic distance d falls below a predefined threshold d_{thresh} (e.g., 20 cm), and
- The binary filter outputs from both IR sensors indicate persistent obstacle presence.

This decision logic is summarized in Algorithm 11. It enhances detection robustness by leveraging the strengths of both sensing modalities: the ultrasonic sensor's range accuracy and the IR sensors' fast response to proximity.

Algorithm 11 Front Obstacle Detection Logic

```

1: procedure ISFRONTOBSTACLEDETECTED
2:    $isDistanceCritical \leftarrow (usonicDistanceValue < d_{\text{thresh}})$ 
3:   if  $isDistanceCritical$  and  $irLeftIsObstacle$  and  $irRightIsObstacle$  then
4:     return true ▷ Obstacle detected in front
5:   else
6:     return false ▷ No obstacle detected
7:   end if
8: end procedure

```

9. Communication Hub

The ESP32-S3-DevKitC-1 board, shown in Fig. 4, was used as the central communication and control hub, enabling efficient data exchange between the two-wheeled mobile base, the ToF sensor array, and external systems such as computers, microcontrollers, mobile devices, or cloud services, as illustrated in Fig. 8.

9.1. Communication with the Mobile Base

A custom communication protocol was developed to manage the exchange of control commands and telemetry data, which is structured around well-defined command and telemetry packets, implemented in `comm.hpp`. These packets support multiple data formats, ensuring compatibility with various communication protocols. Users can send commands to the mobile base and receive real-time telemetry feedback from the mobile base by using the provided member functions `comm.hpp` using communication protocols such as Inter-Integrated Circuit (I2C) and Universal Asynchronous Receiver-Transmitter (UART).

Inter-Integrated Circuit (I2C) communication is used for short-range, two-wire serial communication. The robot operates as an I2C slave with a predefined address (`SLAVE_ADDR = 8`), allowing it to receive commands and transmit telemetry data over the I2C bus from the I2C master. The functions responsible for sending and receiving telemetry data are listed in Table 2.

Table 2: I2C Communication Methods

Packet Type	Function	Description
Command Packets	<code>sendI2CBytes(addr)</code>	Sends command data as raw bytes to the address.
	<code>sendI2CASCII(addr)</code>	Sends command data as ASCII to the address.
	<code>readI2CBytes(addr)</code>	Reads command data as raw bytes from the address.
	<code>readI2CASCII(addr)</code>	Reads command data as ASCII from the address.
Telemetry Packets	<code>sendI2CBytes()</code>	Sends telemetry data as raw bytes.
	<code>sendI2CASCII()</code>	Sends telemetry data as ASCII.
	<code>readI2CBytes(addr)</code>	Reads telemetry data as raw bytes from the address.
	<code>readI2CASCII(addr)</code>	Reads telemetry data as ASCII from the address.

The Universal Asynchronous Receiver-Transmitter (UART) protocol is used for serial communication. The UART communication is initialized at a baud rate of 9600, which is used for the Bluetooth module used in this project [33]. The functions responsible for sending and receiving telemetry data are listed in Table 3. For this project, UART communication was used as it was found to be more reliable.

9.1.1. Sending Commands

Users can send commands to control the robot's movement, rotation, or stop function. Commands follow a predefined structured format and can be transmitted via either I2C or UART. The available commands are listed in Table 4.

ASCII Format: The commands sent in ASCII is formatted as a comma-separated string:

<command>,<command_value>,<command_speed>

Table 3: UART Communication Methods

Packet Type	Function	Description
Command Packets	<code>readUartBytes()</code>	Reads command data as raw bytes.
	<code>readUartASCII()</code>	Reads command data as an ASCII string.
	<code>sendUartBytes()</code>	Sends command data as raw bytes.
	<code>sendUartASCII()</code>	Sends command data as an ASCII string.
Telemetry Packets	<code>sendUartBytes()</code>	Sends telemetry data as raw bytes.
	<code>sendUartASCII()</code>	Sends telemetry data as an ASCII string.
	<code>readUartBytes()</code>	Reads telemetry data as raw bytes.
	<code>readUartASCII()</code>	Reads telemetry data as an ASCII string.

Table 4: List of Commands and Corresponding Values

Command	Value	Description
Stop	0	Stops the robot's movement. <i>Range: -32,768 to 32,767</i>
Move	1	Moves forward or backward (in cm) at a given speed. <i>Range: -32,768 to 32,767</i>
Rotate	2	Rotates the robot by an angle (in degrees) at a given speed. <i>Range: 0 to 359</i>
INVALID	3	Represents an invalid or unrecognized command. <i>Range: -32,768 to 32,767</i>

Example Command:

- Stop the robot: 0,0,0
- Move forward 100 cm at 50% speed: 1,100,50
- Rotate 90° at 30 speed%: 2,90,30

9.1.2. Receiving Telemetry Data

The robot continuously monitors its state and environment using onboard sensors. This data is packaged into a structured format and transmitted back to the user for real-time monitoring and feedback. Telemetry data includes:

- **Yaw Angle:** The robot's current orientation around the y -axis shown in Fig. 32 in degrees.
- **Distance Travelled:** The total distance travelled by the robot in centimetres.
- **Ultrasonic Distance:** The distance to the nearest obstacle, as measured by the ultrasonic sensor in centimetres.
- **Left IR Sensor:** Digital signal indicating detection of a persistent obstacle by the left sensor.
- **Right IR Sensor:** Digital signal indicating detection of a persistent obstacle by the right sensor.
- **Left Motor Encoder Pulses:** Total number of pulses recorded by the left wheel's encoder.
- **Right Motor Encoder Pulses:** Total number of pulses recorded by the right wheel's encoder.

ASCII Format: The telemetry data received in ASCII is formatted as a comma-separated string:

```
<yaw_angle>,<distance_traveled>,<ultrasonic_distance>,<left_IR>,<right_IR>,<left_encoder>,<right_encoder>
```

Example Output:

```
45,200,30,1,0,1540,1572
```

This indicates:

- A yaw angle of 45 degrees
- A total distance travelled of 200 cm
- An ultrasonic measures an obstacle at a distance of 30 cm ahead
- The left IR sensor detects an obstacle (1)
- The right IR sensor does not detect an obstacle (0)
- 1540 total encoder pulses on the left motor
- 1572 total encoder pulses on the right motor

9.2. Communication with TerraRanger Multiflex

The TerraRanger Multiflex communicates via UART, transmitting raw data for all eight sensors in ASCII format. The data string begins with the prefix 'MF' and is separated by tab characters, as shown below:

```
MF\t<Sensor 1>\t<Sensor 2>\t<Sensor 3>\t<Sensor 4>\t<Sensor 5>\t<Sensor 6>\t<Sensor 7>\t<Sensor 8>\r\n
```

In this format, each '<Sensor n>' represents the distance measured by the corresponding sensor in millimeters. The actual raw data received over UART from the TerraRanger Multiflex is illustrated in Fig. 29.

	1	5	10	15	20	25	30	35
MF	37	39	68	72	51	37	45	34
MF	36	38	68	72	50	37	43	35
MF	37	38	68	73	51	37	43	36
MF	37	39	69	70	51	36	44	35
MF	35	38	69	71	50	36	45	36
MF	35	38	68	72	50	36	45	35
MF	36	39	67	71	51	36	44	34

Figure 29: The received raw data from the TerraRanger Multiflex via UART.

9.3. Communication with External Systems

The ESP32-S3 microcontroller is configured to operate in Soft Access Point (SoftAP) mode, enabling it to create a dedicated Wi-Fi network. This configuration allows external devices, such as a remote PC or handheld controller, to connect directly to the ESP32-S3 without requiring an intermediary router. Once connected, the ESP32-S3 facilitates two-way communication between the robot and the external system by transmitting telemetry data and receiving control commands, as illustrated in Fig. 8.

9.3.1. Telemetry Transmission

To support real-time monitoring, the ESP32-S3 establishes a persistent TCP socket connection with a remote PC server. Sensor data—including measurements from the TerraRanger Multiflex (ToF sensors) and the mobile base—is read via UART interfaces. The received ASCII strings are then transmitted to the remote PC server. The system ensures low-latency updates and verifies the connection status before each transmission to maintain data integrity. Fig. 30 illustrates the real-time visualization of the received telemetry in a polar plot.

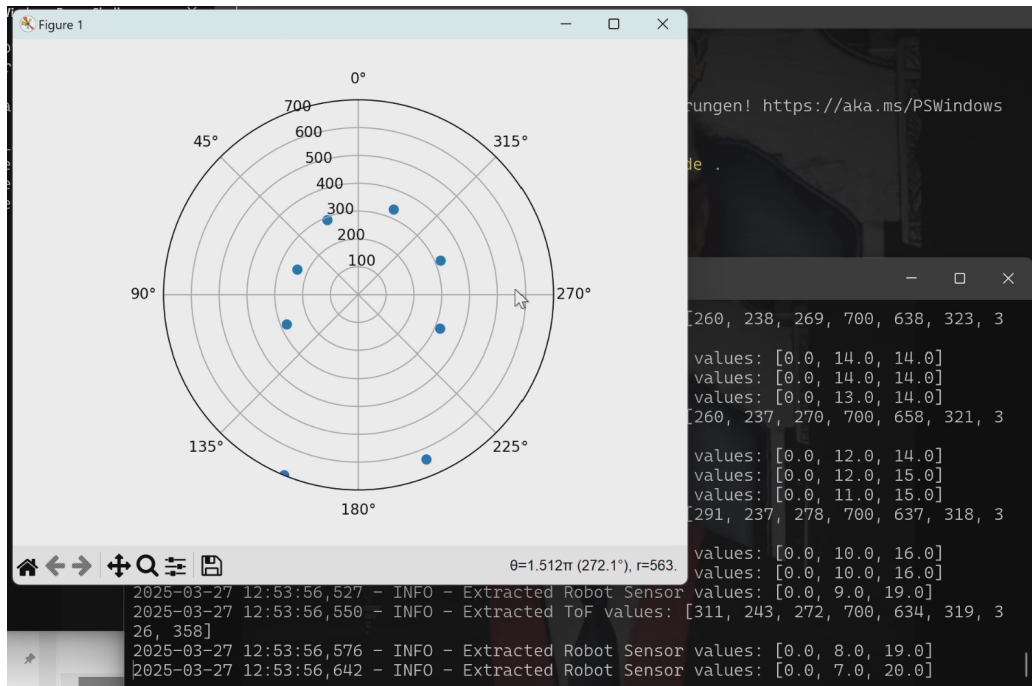


Figure 30: Real-time sensor data visualization in a polar plot.

9.3.2. Remote Command Reception and Execution

The ESP32-S3 also supports command reception over TCP from a connected external device. This mechanism allows remote users to control the mobile robot in real time. The workflow for command handling is described below:

1. **TCP Listening:** The ESP32-S3 continuously monitors its TCP server for incoming client connections.

2. **Command Reception:** Once a client is connected, the ESP32-S3 reads the incoming ASCII-encoded command string. This string typically contains a command type (e.g., MOVE, ROTATE, STOP), a command value (e.g., angle or distance), and an associated speed parameter.
3. **Parsing and Validation:** The received string is parsed to extract individual parameters. These parameters are then encapsulated into a structured data packet.
4. **UART Transmission to Robot:** The formatted command packet is transmitted to the mobile base via UART.
5. **Command Execution:** The ESP32-S3 supports the reception of remote control commands transmitted over a TCP socket in the form of structured ASCII strings. The available commands are listed in Table 4. These commands are formatted as comma-separated values, representing the action to be performed, its magnitude, and the execution speed. A typical command string follows the pattern:

`<COMMAND>,<VALUE>,<SPEED>`

6. **Client Disconnection:** After command execution, the ESP32-S3 acknowledges the command and closes the client connection.

This bidirectional communication model enables seamless remote control and continuous monitoring, making it suitable for autonomous robotic applications and teleoperation scenarios.

10. Conclusion

This project aimed to design and implement a cost-effective, two-wheeled self-balancing robot integrated with a Time-of-Flight (ToF) sensor array for 2D environmental mapping and remote control via Wi-Fi. The final assembly of the robot is depicted in Fig. 31.

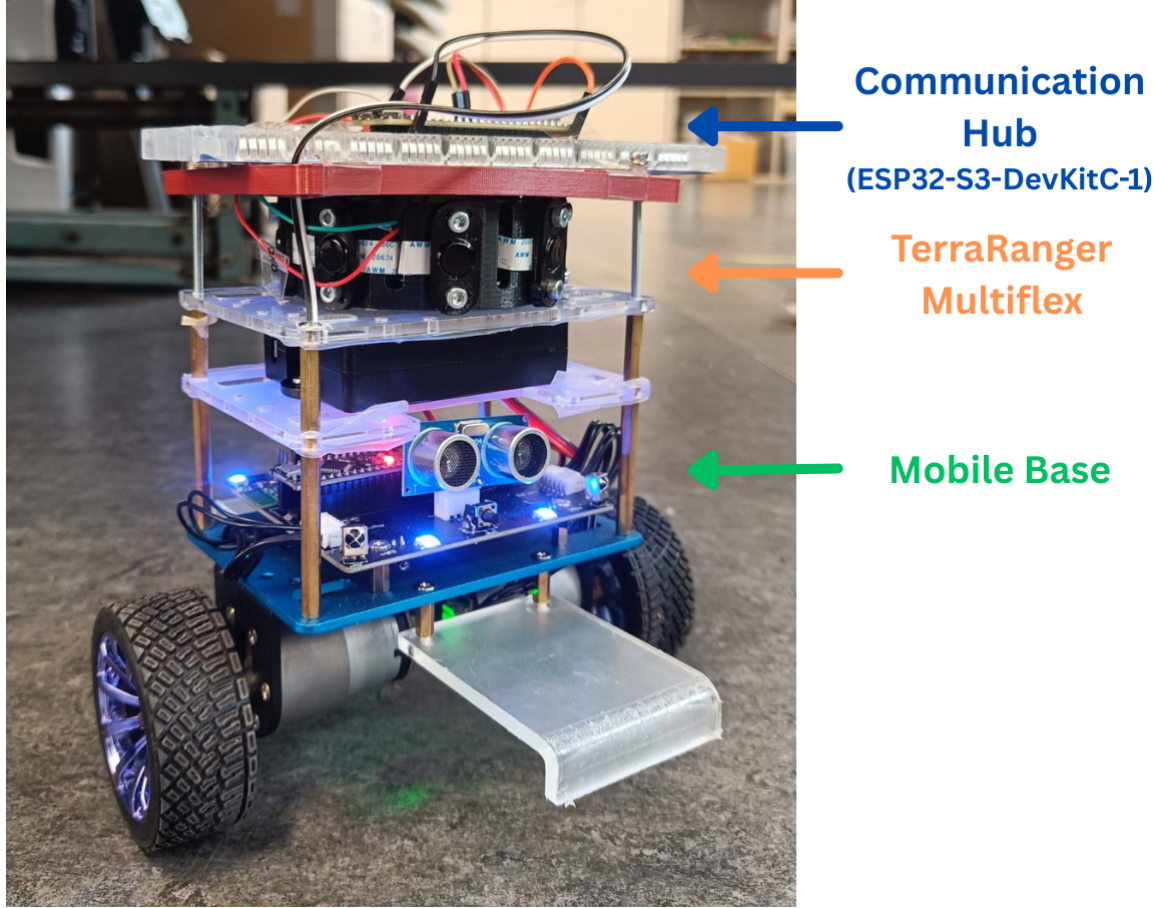


Figure 31: Final assembly of the remotely operated two-wheeled robot with TerraRanger Multiflex.

Control strategies were implemented using a multi-rate PID controller and Kalman filtering to ensure robust balance and motion control. The communication framework utilized TCP sockets over SoftAP Wi-Fi mode, enabling real-time telemetry streaming and command reception from remote systems. This architecture effectively demonstrates how low-cost components and open-source development environments can be harnessed to build intelligent robotic platforms capable of real-world interaction and monitoring.

Overall, the project successfully achieved its core objectives by integrating balancing control, environmental sensing, and networked communication into a unified robotic platform. The result is a versatile and extensible robot suitable for future enhancements such as autonomous navigation, SLAM integration, and multi-agent collaboration. All source code and documentation are publicly available to facilitate reproducibility and continued research in this domain.

11. Future Work and Improvements

The current system establishes a robust framework for real-time monitoring and control of a self-balancing robot equipped with a TeraRanger Multiflex sensor array. Future enhancements will focus on improving autonomy, robustness, and intelligence.

Advanced control strategies such as Model Predictive Control (MPC), adaptive PID, or reinforcement learning will be investigated to improve stability and responsiveness under dynamic conditions. For full autonomy, integration of Simultaneous Localization and Mapping (SLAM) and path-planning algorithms will enable real-time navigation and obstacle avoidance in complex environments.

Sensor capabilities will be extended by incorporating vision systems and additional IMUs, with multi-sensor fusion improving spatial awareness and decision-making. ROS2 integration is planned to enable modular software development, interoperability with other robotic platforms, and enhanced scalability.

Deploying lightweight machine learning models on the robot's microcontroller could enable intelligent behavior and real-time adaptation to dynamic environments. This would allow the robot to make decisions based on sensor data locally, reducing reliance on remote processing and improving overall responsiveness.

Long-term goals include multi-robot coordination through swarm robotics, requiring decentralized communication and task allocation frameworks. Finally, embedding lightweight machine learning models on-device will facilitate local inference and adaptive behavior, reducing reliance on external computation and enhancing system responsiveness.

Appendices

A. Calculation of the system's transfer functions

Following modelling approach by Prasad and Tyagi [20], and MathWorks [8], we simplify the model and represent it as an inverted pendulum connected to a wheel that moves in one direction. The wheelbase of robot is assumed to be sliding on a frictionless surface along x -axis, as shown in Fig. 5.

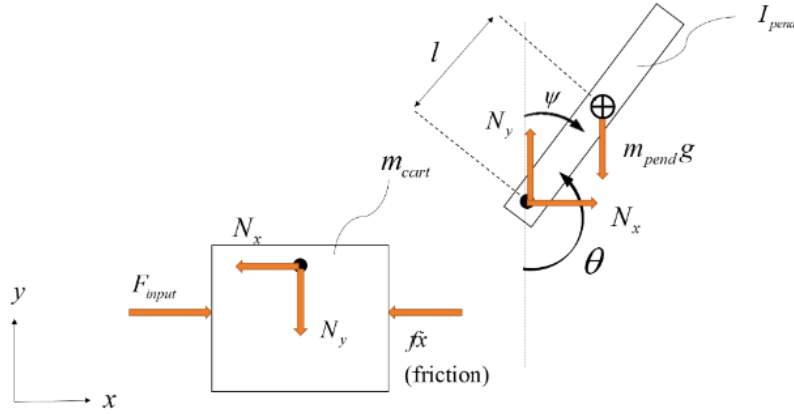


Figure 32: Free-body diagram of the simplified system with cart and pendulum [35]. Where, x axis is along the horizontal axis and y axis is along the vertical axis and $\psi = \pi - \theta$.

A.1. Equation for the cart

The sums forces acting in the x direction on the cart is given by

$$F_{input} = m_{cart}\ddot{x} + f\dot{x} + N_x \quad (27)$$

where N_x is the contact force acting along the x -axis between cart and pendulum, f is a constant representing friction, m_{cart} denotes mass of the cart, and F_{input} represents the applied force as shown in Fig. 32.

A.2. Equation for the the pendulum

The sum of forces acting in the x direction on the pendulum is given by

$$N_x = m_{pend}\ddot{x} + m_{pend}l\ddot{\theta}\cos\theta - m_{pend}l\dot{\theta}^2\sin\theta \quad (28)$$

where θ is the angle between a vertical y -axis and the pendulum, and m_{pend} is the mass and l is distance to the centre of mass as shown in Fig. 32. The sum of all forces perpendicular to the pendulum is

$$N_y\sin\theta + N_x\cos\theta = m_{pend}g\sin\theta - m_{pend}l\ddot{\theta} + m_{pend}\dot{\theta}^2\cos\theta \quad (29)$$

where N_y is the normal force acting along the y direction and g is the value for gravitational acceleration.

The sum of torques acting on the centre of pendulum is given by

$$-N_y l \sin \theta - N_x l \cos \theta = I_{pend} \ddot{\theta} \quad (30)$$

where I_{pend} denotes pendulum's moment of inertia.

A.3. Combining the equations

By combining equations (28) and (29) gives the following equation,

$$F_{input} = (m_{cart} + m_{pend})\ddot{x} + f\dot{x} + m_{pend}l\ddot{\theta} \cos \theta - m_{pend}l\dot{\theta}^2 \sin \theta \quad (31)$$

By combining equations (30) and (31) gives the following equation,

$$(I_{pend} + m_{pend}l^2)\ddot{\theta} + m_{pend}gl \sin \theta = -m_{pend}l\ddot{x} \cos \theta \quad (32)$$

A.4. Linearising the equations

If the pitch angle ψ represent the deviation of pendulum from equilibrium, as shown in Fig. 32, the following approximations for small deviations can be made

$$\cos \theta = \cos(\pi + \psi) \approx -1 \quad (33)$$

$$\sin \theta = \sin(\pi + \psi) \approx -\psi \quad (34)$$

$$\dot{\theta}^2 = \dot{\psi}^2 \approx 0 \quad (35)$$

Using approximations (33), (34) and (35) the non-linear equations (31) and (32) can be linearised to following approximated equations

$$(I_{pend} + m_{pend}l^2)\ddot{\psi} - m_{pend}gl\psi = m_{pend}l\ddot{x} \quad (36)$$

$$u_{input} = (m_{cart} + m_{pend})\ddot{x} + f\dot{x} - m_{pend}l\ddot{\psi} \quad (37)$$

where u_{input} is control effort of the system and replaces F_{input} in equation (31).

A.5. Laplace transform

The Laplace transforms of equations (36) and (37) are given by

$$(I_{pend} + m_{pend}l^2)\Psi(s)s^2 - m_{pend}gl\Psi(s) = m_{pend}lX(s)s^2 \quad (38)$$

$$U_{input}(s) = (m_{cart} + m_{pend})X(s)s^2 + fX(s)s - m_{pend}l\Psi(s)s^2 \quad (39)$$

Solving equation (38) for $X(s)$ gives

$$X(s) = \left[\frac{I_{pend} + m_{pend}l^2}{m_{pend}l} - \frac{g}{s^2} \right] \Psi(s) \quad (40)$$

Substituting equation (40) into (39) gives

$$\begin{aligned} U_{input}(s) = & (m_{cart} + m_{pend}) \left[\frac{I_{pend} + m_{pend}l^2}{m_{pend}l} - \frac{g}{s^2} \right] \Psi(s)s^2 \\ & + f \left[\frac{I_{pend} + m_{pend}l^2}{m_{pend}l} - \frac{g}{s^2} \right] \Psi(s)s - m_{pend}l\Psi(s)s^2 \end{aligned} \quad (41)$$

By rearranging equation (41) the transfer function $G_\Psi(s)$ that describes the cart position $\Psi(s)$ is given by

$$G_\Psi(s) = \frac{\Psi(s)}{U_{input}(s)} \quad (42)$$

$$\Psi(s) = \frac{\frac{m_{pend}l}{q}s}{\underbrace{s^3 + \frac{f(I_{pend}+m_{pend}l^2)}{q}s^2 - \frac{(m_{cart}+m_{pend})m_{pend}gl}{q}s - \frac{fm_{pend}gl}{q}}_{G_\Psi(s)}} U_{input}(s) \quad (43)$$

Where,

$$q = [(m_{cart} + m_{pend})(I_{pend} + m_{pend}l^2) - (m_{pend}l)^2] \quad (44)$$

The transfer function $G_x(s)$ that describes the cart position $X(s)$ is given by

$$G_x(s) = \frac{X(s)}{U_{input}(s)} \quad (45)$$

$$X(s) = \frac{\frac{(I_{pend}+m_{pend}l^2)s^2 - gm_{pend}l}{q}}{\underbrace{s^4 + \frac{f(I_{pend}+m_{pend}l^2)}{q}s^3 - \frac{(m_{cart}+m_{pend})m_{pend}gl}{q}s^2 - \frac{fm_{pend}gl}{q}s}_{G_x(s)}} U_{input}(s) \quad (46)$$

A.6. State Space Modelling

The system in state space form is represented as

$$\begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\psi} \\ \ddot{\psi} \end{bmatrix} = A \begin{bmatrix} x \\ \dot{x} \\ \psi \\ \dot{\psi} \end{bmatrix} + B u_{input} \quad (47)$$

$$y = C u_{input} \quad (48)$$

The system matrices A , B and C can be derived by solving Eq. 36 and Eq. 37:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & \frac{-(I_{pend} + m_{pend}l^2)f}{I_{pend}(m_{cart} + m_{pend}) + m_{cart}m_{pend}l^2} & \frac{m_{pend}^2gl^2}{I_{pend}(m_{cart} + m_{pend}) + m_{cart}m_{pend}l^2} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{-m_{pend}lf}{I_{pend}(m_{cart} + m_{pend}) + m_{cart}m_{pend}l^2} & \frac{m_{pend}gl(m_{cart} + m_{pend})}{I_{pend}(m_{cart} + m_{pend}) + m_{cart}m_{pend}l^2} & 0 \end{bmatrix} \quad (49)$$

$$B = \begin{bmatrix} 0 \\ \frac{I_{pend} + m_{pend}l^2}{I_{pend}(m_{cart} + m_{pend}) + m_{cart}m_{pend}l^2} \\ 0 \\ \frac{m_{pend}l}{I_{pend}(m_{cart} + m_{pend}) + m_{cart}m_{pend}l^2} \end{bmatrix} \quad (50)$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (51)$$

B. Calculation for Kalman Filter

Using Kalman Filtering technique discussed by Pengfei Gui et al. [15] we can derived following mathematical equations for IMU data fusion. The state equation and the estimate equation of the system based on the derivative relation of slant angle and angular velocity can be establish as following,

$$\begin{aligned} \psi_{new} &= \psi_{previous} + (\dot{\psi} - \dot{\psi}_b) * \Delta t \\ y &= \psi \end{aligned} \quad (52)$$

Then state-space representation of the system is given by,

$$\begin{aligned} \dot{x}(t) &= A x(t) + B u_{input} \\ y(t) &= C x(t) + D u_{input} \end{aligned} \quad (53)$$

Where,

- **State Vector:** This vector encapsulates the internal state of the system at time t . In this case, it is defined as:

$$x(t) = \begin{bmatrix} \psi \\ \dot{\psi}_b \end{bmatrix} \quad (54)$$

Where $\dot{\psi}_b$ denotes the bias or drift error of the gyroscope.

- **Control Vector:** This vector represents the external input given to the control-input model at step k . In this case,

$$u_{input} = \dot{\psi}_{gyro} \quad (55)$$

Where, $\dot{\psi}_{gyro}$ is the pitch angular velocity measured by gyroscope.

- **State Transition Model:** This matrix describes how the state evolves over time. It is defined as:

$$A = \begin{bmatrix} 0 & -1 \\ 0 & 0 \end{bmatrix} \quad (56)$$

The first row indicates that the angle is updated based on its previous value, and the second row shows that the gyroscope bias remains constant in this model.

- **Control Input Model:** This matrix relates the control inputs to the state. In this case, it is defined as:

$$B = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (57)$$

This indicates that the angle is the only direct control inputs affecting the state in this model.

- **Measurement Model:** This matrix maps the state vector to the measurement output. It is defined as:

$$C = \begin{bmatrix} 1 & 0 \end{bmatrix} \quad (58)$$

This means that the measurement output directly reflects the angle, with no contribution from the gyroscope bias.

- **Feed-forward Model:** This matrix relates the control input directly to the measurement output. In this case, it is defined as:

$$D = 0 \quad (59)$$

This indicates that there is no direct influence of the control input on the measurement output.

B.1. Time-Discrete Kalman Filter Model

The Kalman filter model assumes the true state x_k at time k is evolved from the state x_{k-1} at $k - 1$ according to,

$$x_k = G(\Delta t) x_{k-1} + H(\Delta t) u_{k-1} + d_k \quad (60)$$

Similarly, at time k an observation (or measurement) z_k of the true state x_k is given by,

$$z_k = C_k x_k + n_k \quad (61)$$

Where,

- **Discrete State Vector:**

$$x_k = \begin{bmatrix} \psi_k \\ \dot{\psi}_{b,k} \end{bmatrix} \quad (62)$$

Where ψ_k represents the measured angle of the system and $\dot{\psi}_{b,k}$ denotes the error or bias of the gyroscope.

- **State Transition Model:** The State transition matrix G propagates the state x_{k-1} at sample $k - 1$ to the state at sample k . It captures all of the relevant internal dynamics of the system by describing how each state moves over one sample time Δt . It can be derived from State Transition Matrix A by applying the principles of discretization as following,

$$G(\Delta t) = e^{A\Delta t} \quad (63)$$

Because matrix A is a Nilpotent matrix therefore,

$$G(\Delta t) = I + A\Delta t = \begin{bmatrix} 1 & -\Delta t \\ 0 & 1 \end{bmatrix} \quad (64)$$

- **Control Input Model:** The control input u_{input} into the system is accounted for with the control matrix H

$$H(\Delta t) = B \int_0^{\Delta t} e^{A\sigma} d\sigma \quad (65)$$

Because matrix A is a Nilpotent matrix therefore,

$$H(\Delta t) = \begin{bmatrix} \Delta t \\ 0 \end{bmatrix} \quad (66)$$

- **Measurement Model:** The measurement matrix C for discrete model remains the same,

$$C_k = \begin{bmatrix} 1 & 0 \end{bmatrix} \quad (67)$$

- **Process/System Noise:**

d_k represents process noise which is Gaussian distributed with a zero mean and with covariance Q_k at the time step k ,

$$d_k \sim N(0, Q_k) \quad (68)$$

In this case we will consider the estimate of the bias and the accelerometer to be independent, so it's actually just equal to the variance of the estimate of the accelerometer and bias. So the process noise covariance matrix can be written as,

$$Q_k = \begin{bmatrix} Q_\psi & 0 \\ 0 & Q_{\psi_b} \end{bmatrix} * \Delta t \quad (69)$$

Where covariance matrix Q_k depends on the current time k , so the accelerometer variance Q_ψ and the variance of the bias Q_{ψ_b} is multiplied by the delta time.

- **Measurement Noise:** $n(k)$ represents measurement noise at time step k which is Gaussian distributed as well with a zero mean and R_k as the covariance,

$$n_k \sim N(0, R_k) \quad (70)$$

In this case, R_k the measurement noise matrix is equal to the variance of the measurement, since the covariance of same variable is equal to the variance. We can define R_k as,

$$R_k = E \begin{bmatrix} n_k & n_k^T \end{bmatrix} = var(n_k) \quad (71)$$

Where, E is the expectation operator. We can be assumed that the measurement noise is the same and does not depend on the time k ,

$$var(n_k) = var(n) = R_\psi \quad (72)$$

Where, R_ψ is constant and provided by the manufacturer of sensor.

B.2. Implementing Kalman Filter

The Kalman filter is a recursive estimator, which requires only the estimated state from the previous time step $k - 1$ and the current measurement to compute the estimate state for the current time step k .

B.2.1. Predicted State Estimate

A priori estimate of the angle $\hat{x}_{k|k-1}$ is made according to following relationship,

$$\hat{x}_{k|k-1} = G \hat{x}_{k-1|k-1} + H \dot{\psi}_{gyro,k} \quad (73)$$

Where, $\hat{x}_{k-1|k-1}$ is the estimate of the state at previous step $k - 1$, and $\dot{\psi}_{gyro,k}$ is the angular velocity measured by the gyroscope at current step k .

B.2.2. Predicted Error Covariance

The state covariance matrix P is given by,

$$P = \begin{bmatrix} P_{00} & P_{01} \\ P_{10} & P_{11} \end{bmatrix} \quad (74)$$

The matrix P reflects the uncertainties in the angle estimate (P_{00}), the gyroscope bias (P_{11}), and the cross-covariance terms (P_{01} , P_{10}). The a priori error covariance matrix $P_{k|k-1}$ can be estimated based on the previous error covariance matrix $P_{k-1|k-1}$ as,

$$P_{k|k-1} = G P_{k-1|k-1} G^T + Q_k \quad (75)$$

This further expands to,

$$P_{k|k-1} = \begin{bmatrix} P_{00} & P_{01} \\ P_{10} & P_{11} \end{bmatrix}_{k|k-1} = \begin{bmatrix} P_{00}^- + [(Q_\psi + \Delta t P_{11} - P_{01}^- - P_{10}^-) * \Delta t] & P_{01}^- - (P_{11}^- * \Delta t) \\ P_{10}^- - (P_{11}^- * \Delta t) & P_{11}^- + (Q_{\dot{\psi}_b} * \Delta t) \end{bmatrix} \quad (76)$$

Where the negative sign (e.g. " - ") on top of P_{00}^- , P_{10}^- , P_{11}^- and P_{01}^- represents that these parameters were calculated using previous step $k - 1$ values. The initial state covariance matrix is defined as,

$$P_0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad (77)$$

B.2.3. Innovation Residual

The innovation residual $\tilde{y}_k$ is the difference between the output of the sensor, z_k , and the predicted state, $\hat{x}_{k|k-1}$, rotated into the sensor output frame with C_k

$$\tilde{y}_k = z_k - C_k \hat{x}_{k|k-1} \quad (78)$$

B.2.4. Innovation Covariance

The innovation covariance S_k is the sum of the prediction covariance P , and the measurement covariance R ,

$$\begin{aligned} S_k &= C P_{k|k-1} C^T + R_k \\ &= \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} P_{00} & P_{01} \\ P_{10} & P_{11} \end{bmatrix}_{k|k-1} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + R_\psi \\ S_k &= P_{00,k|k-1} + R_\psi \end{aligned} \quad (79)$$

B.2.5. Optimal Kalman Gain

We can find optimal Kalman gain matrix K_k as,

$$K_k = P_{k|k-1} C_k^T S_k^{-1} \quad (80)$$

It determines much the state estimate should be adjusted based on the measurement residual. It balances the uncertainty in the state estimate and the measurement noise. The Kalman gain K_k is computed as,

$$\begin{aligned} K_k &= P_{k|k-1} C^T S_k^{-1} \\ &= \begin{bmatrix} P_{00} & P_{01} \\ P_{10} & P_{11} \end{bmatrix}_{k|k-1} \begin{bmatrix} 1 \\ 0 \end{bmatrix} S_k^{-1} \\ &= \begin{bmatrix} P_{00} \\ P_{10} \end{bmatrix}_{k|k-1} (P_{00} + R_\psi)^{-1} \\ K_k &= \begin{bmatrix} K_0 \\ K_1 \end{bmatrix} = \begin{bmatrix} \frac{P_{00}}{P_{00}+R_\psi} \\ \frac{P_{10}}{P_{00}+R_\psi} \end{bmatrix} \end{aligned} \quad (81)$$

B.2.6. Corrected State Estimate

The corrected state estimate is somewhere between the predicted state $x_{k|k-1}$ and the measurement $\tilde{y}_k$, and is calculated as,

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k \tilde{y}_k \quad (82)$$

B.2.7. Corrected Estimate Covariance

The estimate covariance matrix is corrected as,

$$\begin{aligned} P_{k|k} &= (I - K_k C) P_{k|k-1} \\ &= \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \begin{bmatrix} K_0 \\ K_1 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} \right) \begin{bmatrix} P_{00} & P_{01} \\ P_{10} & P_{11} \end{bmatrix}_{k|k-1} = \begin{bmatrix} 1 - K_0 & 0 \\ -K_1 & 1 \end{bmatrix} \begin{bmatrix} P_{00} & P_{01} \\ P_{10} & P_{11} \end{bmatrix}_{k|k-1} \\ &= \begin{bmatrix} P_{00} - K_0 \cdot P_{00} & P_{01} - K_0 \cdot P_{01} \\ P_{10} - K_1 \cdot P_{00} & P_{11} - K_1 \cdot P_{01} \end{bmatrix} \end{aligned} \quad (83)$$

References

- [1] I. I. F. o. Robotics. “Record of 4 million robots in factories worldwide”, IFR International Federation of Robotics, Accessed: Apr. 7, 2025. [Online]. Available: <https://ifr.org/ifr-press-releases/news/record-of-4-million-robots-working-in-factories-worldwide>.
- [2] C. E. Staff. “Autonomous mobile robot market expanding at record pace”, Control Engineering, Accessed: Apr. 8, 2025. [Online]. Available: <https://www.controleng.com/autonomous-mobile-robot-market-expanding-at-record-pace/>.
- [3] S. Wagner, “2d materials for piezoresistive strain gauges and membrane based nanoelectromechanical systems”, Ph.D. dissertation, Dissertation, RWTH Aachen University, 2018, 2018.
- [4] “Global MEMS market size, share, analysis-2032”, Spherical Insights, Accessed: Apr. 8, 2025. [Online]. Available: <https://www.sphericalinsights.com/reports/mems-market>.
- [5] M. Bjelonic, V. Klemm, J. Lee, and M. Hutter, “A survey of wheeled-legged robots”, in *Robotics in Natural Settings*, J. M. Cascalho, M. O. Tokhi, M. F. Silva, A. Mendes, K. Goher, and M. Funk, Eds., Cham: Springer International Publishing, 2023, pp. 83–94.
- [6] D. Kim, W. Chung, and S. Park, “Practical motion planning for car-parking control in narrow environment”, *IET control theory & applications*, vol. 4, no. 1, pp. 129–139, 2010.
- [7] ELEGOO Official, *Elegoo tumbler self-balancing robot car v1.1/v1.0 tutorial*, Accessed: 2025-02-15. [Online]. Available: <https://www.elegoo.com/blogs/arduino-projects/elegoo-tumbler-self-balancing-robot-car-tutorial>.
- [8] MATLAB, *Inverted pendulum: System modeling*, Accessed: 2025-02-15. [Online]. Available: <https://ctms.engin.umich.edu/CTMS/index.php?example=InvertedPendulum§ion=SystemModeling>.
- [9] N. S. Nise, *Control systems engineering*. John Wiley & Sons, 2020.
- [10] K. M. Passino, S. Yurkovich, and M. Reinfrank, *Fuzzy control*. Addison-wesley Reading, MA, 1998, vol. 42.
- [11] A. Abdelgawad, T. Shohdy, and A. Nada, “Model-and data-based control of self-balancing robots: Practical educational approach with labview and arduino”, *IFAC-PapersOnLine*, vol. 58, no. 9, pp. 217–222, 2024.
- [12] R. E. Kalman, “A new approach to linear filtering and prediction problems”, 1960.
- [13] P. Chotikunnan et al., “Comparative analysis of sensor fusion for angle estimation using kalman and complementary filters”, *International Journal of Robotics and Control Systems*, vol. 5, no. 1, pp. 1–21, 2024. [Online]. Available: <https://www.pubs2.ascee.org/index.php/IJRCS/article/view/1674>.
- [14] H.-Q.-T. Ngo, T.-P. Nguyen, V.-N.-S. Huynh, T.-S. Le, and C.-T. Nguyen, “Experimental comparison of complementary filter and kalman filter design for low-cost sensor in quadcopter”, in *2017 International Conference on System Science and Engineering (ICSSE)*, IEEE, 2017, pp. 488–493.

- [15] P. Gui, L. Tang, and S. Mukhopadhyay, “Mems based imu for tilting measurement: Comparison of complementary and kalman filter based data fusion”, in *2015 IEEE 10th Conference on Industrial Electronics and Applications (ICIEA)*, 2015, pp. 2004–2009.
- [16] Mouser Electronics, *Teraranger multiflex key features applications*, <https://mouser.com/datasheet/2/944/MultiflexSpecificationSheet-1880070.pdf>, Accessed: 10 April 2025.
- [17] *ESP32-S3-DevKitC-1*, [Online; accessed 10. Apr. 2025], 2025. [Online]. Available: https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/user_guide.html.
- [18] Thingiverse, *Teraranger multiflex circular mount*, Accessed: 2025-02-15. [Online]. Available: <https://www.thingiverse.com/thing:2311818>.
- [19] *ESP32-S3-DEVKITC-1-N8R8 | DigiKey Electronics*, [Online; accessed 10. Apr. 2025], Apr. 2025. [Online]. Available: <https://www.digikey.de/en/products/detail/espressif-systems/ESP32-S3-DEVKITC-1-N8R8/15295894>.
- [20] L. B. Prasad, B. Tyagi, and H. O. Gupta, “Optimal control of nonlinear inverted pendulum system using pid controller and lqr: Performance analysis without and with disturbance input”, *International Journal of Automation and Computing*, vol. 11, no. 6, pp. 661–670, 2014. [Online]. Available: <https://doi.org/10.1007/s11633-014-0818-1>.
- [21] Microchip, *Atmega328p*, Accessed: 2024-11-23. [Online]. Available: <https://www.microchip.com/en-us/product/ATmega328p>.
- [22] Arduino S.r.l., *Arduino nano*, Accessed: 2025-02-09. [Online]. Available: <https://store.arduino.cc/en-de/products/arduino-nano>.
- [23] E. Official, *Elegoo smart robot car kit v4.0 tutorial*, Accessed: 2025-02-09. [Online]. Available: <https://www.elegoo.com/blogs/arduino-projects/elegoo-smart-robot-car-kit-v4-0-tutorial>.
- [24] InvenSense, *Mpu-6050*, Accessed: 2024-11-23. [Online]. Available: <https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/>.
- [25] Toshiba, *Tb6612fng*, Accessed: 2024-11-23. [Online]. Available: <https://toshiba.semicon-storage.com/us/semiconductor/product/motor-driver-ics/brushed-dc-motor-driver-ics/detail.TB6612FNG.html>.
- [26] BerryBase - The Maker Shop, *Sparkfun motor treiber, dual tb6612fng, mit headern*, Accessed: 2025-02-09. [Online]. Available: <https://www.berrybase.de/sparkfun-motor-treiber-dual-tb6612fng-mit-headern>.
- [27] Texas Instruments, *Sn74lvc2g14*, Accessed: 2024-11-23. [Online]. Available: <https://www.ti.com/product/SN74LVC2G14>.
- [28] Cadence Design Systems, *Schmitt trigger hysteresis provides noise-free switching and output*, Accessed: 2025-02-09. [Online]. Available: <https://resources.pcb.cadence.com/blog/2021-schmitt-trigger-hysteresis-provides-noise-free-switching-and-output>.

- [29] MicroDCmotors, *37mm 6v 12v 24v dc gear motor w/encoder model nfp-gm37-520-en*, Accessed: 2024-11-23. [Online]. Available: <https://microdcmotors.com/product/6v-12v-geared-dc-electric-motor-with-encoder-nfp-jgb37-520-en>.
- [30] SparkFun Electronics, *Ultrasonic distance sensor - 5v (hc-sr04)*, Accessed: 2025-02-08. [Online]. Available: <https://www.sparkfun.com/ultrasonic-distance-sensor-hc-sr04.html>.
- [31] DigiKey, *Everlight electronics co ltd ir204c-a*, Accessed: 2025-02-08. [Online]. Available: <https://www.digikey.de/de/products/detail/everlight-electronics-co-ltd/IR204C-A/2675568>.
- [32] X-ON Electronics Services, *Irm-56384 everlight*, Accessed: 2025-02-08. [Online]. Available: <https://www.xonelec.com/mpn/everlight/irm56384>.
- [33] epow0.org, *Elegoo bt16 bluetooth uart module*, Accessed: 2025-02-09. [Online]. Available: https://epow0.org/~amki/car_kit/Datasheet/ELEG00%20BT16%20Bluetooth%20UART%20Module.pdf.
- [34] F. F. Rabbany, A. Qurthobi, and A. Suhendi, “Design of self-balancing virtual reality robot using PID control method and complementary filter”, in *2021 IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology (IAICT)*, Bandung, Indonesia: IEEE, Jul. 27, 2021, pp. 15–19. Accessed: Mar. 16, 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/9532576/>.
- [35] D. Sedlar and A. Bosnjak, “Design and control based on the concept of an inverted pendulum”, in *2023 8th International Conference on Smart and Sustainable Technologies (SpliTech)*, 2023, pp. 1–5.
- [36] K. Xu and X.-D. Duan, “Comparative study of control methods of single-rotational inverted pendulum”, in *Proceedings. International Conference on Machine Learning and Cybernetics*, vol. 2, 2002, 776–778 vol.2.
- [37] K. Liu, M. Bai, and Y. Ni, “Two-wheel self-balanced car based on kalman filtering and pid algorithm”, in *2011 IEEE 18th International Conference on Industrial Engineering and Engineering Management*, vol. Part 1, 2011, pp. 281–285.
- [38] F. Sun, Z. Yu, and H. Yang, “A design for two-wheeled self-balancing robot based on kalman filter and lqr”, in *2014 International Conference on Mechatronics and Control (ICMC)*, 2014, pp. 612–616.
- [39] S. Kumar, Vijaydeep, and P. Gupta, “Self-balancing vehicle using kalman filter”, in *2017 International Conference on Communication and Signal Processing (ICCSP)*, 2017, pp. 1496–1499.
- [40] TKJElectronics, *Tkjelectronics kalmanfilter*, Accessed: 2025-02-15. [Online]. Available: <https://github.com/TKJElectronics/KalmanFilter>.
- [41] O. Jamil, M. Jamil, Y. Ayaz, and K. Ahmad, “Modeling, control of a two-wheeled self-balancing robot”, in *2014 International Conference on Robotics and Emerging Allied Technologies in Engineering (iCREATE)*, Islamabad, Pakistan: IEEE, Apr. 2014, pp. 191–199. Accessed: Mar. 16, 2024. [Online]. Available: <http://ieeexplore.ieee.org/document/6828364/>.